

学校代码: 10284
分类号: TP311.5
密 级: 公开
U D C: 004.41
学 号: 522024720008



南京大學

硕士学位论文

论文题目	面向 C 程序切片工具的 蜕变测试 与自动化缺陷检测研究
作者姓名	袁闯
专业学位类别(领域)	电子信息（软件工程领域）
研究方向	程序分析与软件测试
导师姓名	钮鑫涛 助理教授

2026 年 5 月 22 日

答辩委员会主席	张贺 教授
评 阅 人	梁鹏 教授
	杜庆伟 副教授
	葛季栋 副教授

论文答辩日期 2026 年 5 月 22 日

研究生签名:	袁 闯
导师签名:	钮鑫涛

Research on Metamorphic Testing and Automated Defect Detection for C Program Slicing Tools

by
Yuan Chuang

Supervised by
Niu XinTao

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of
MASTER
in
Electronic Information (Major in Software Engineering)



School of Intelligent Software and Engineering
Nanjing University

May 22, 2026

南京大学学位论文原创性声明

本人郑重声明，所提交的学位论文是本人在导师指导下独立进行科学研究工作所取得的成果。除本论文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得南京大学或其他教育机构的学位证书而使用过的材料。对本文的研究做出重要贡献的个人和集体，均已在论文的致谢部分明确标明。本人郑重声明愿承担本声明的法律责任。

研究生签名：袁闯

日期：2026 年 5 月 22 日

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向 C 程序切片工具的蜕变测试与自动化缺陷检测研究

电子信息(软件工程领域)专业 2024 级硕士生姓名：袁闯

指导教师(姓名、职称)：钮鑫涛 助理教授

摘 要

程序切片是程序理解、故障定位与安全分析中的基础技术，其结果质量直接影响上层分析任务的可靠性。对于 C 语言而言，指针与别名、复杂控制流以及过程间传播等特性增加了依赖关系恢复的难度，也使切片工具更容易出现依赖遗漏、冗余保留和语义失真等问题。因此，面向 C 程序切片工具开展自动化可靠性验证，具有重要的研究意义与工程价值。

然而，当前 C 程序切片工具测试仍面临三方面挑战：(1) 测试预言缺失：切片结果通常表现为语句集合、依赖关系或中间表示，缺乏低成本、可自动化的正确性判定依据；(2) 深层语义场景触发不足：传统自动生成方法虽然能够扩大输入空间，但对复杂别名关系、长距离数据流和细粒度控制依赖等场景的定向构造能力仍然有限；(3) 异构结果判定困难：源码级与 IR 级切片工具在输入形式、依赖建模粒度和结果表示方式上存在明显差异，进一步增加了统一组织验证流程与结果判定的难度。

针对上述挑战，本文将蜕变测试思想应用于 C 程序切片工具测试，围绕切片语义设计四类领域专用蜕变关系，将切片结果的正确性判定转化为关系一致性检查，从而缓解切片工具测试中的预言缺失问题。为增强复杂语义场景的触发能力，本文结合 Csmith 随机生成与大语言模型引导生成构造测试程序，以覆盖不同程序结构和复杂依赖场景。在自动化检测层面，本文基于上述蜕变关系设计自动化测试框架，面向源码级与 LLVM IR 级切片工具组织测试程序生成、切片执行、结果判定与缺陷整理等过程。实验共发现并整理出 43 个可复现问题项，其中 Frama-C 为 17 个、dg 为 26 个；Frama-C 中已有 13 个问题项被开发者确认。实验结果表明，基于蜕变测试的检测方式能够用于 C 程序切片工具测试，并能在自动化缺陷检测中发现可复现问题，验证了所提方法的可行性。

关键词：程序切片；蜕变测试；静态分析；C 语言；大语言模型

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on Metamorphic Testing and Automated Defect Detection for C Program Slicing Tools

SPECIALIZATION: Electronic Information (Major in Software Engineering)

POSTGRADUATE: Yuan Chuang

MENTOR: Niu XinTao

ABSTRACT

Program slicing is a fundamental technique in program comprehension, fault localization, and security analysis, and the quality of slicing results directly affects the reliability of downstream analysis tasks. For C programs, features such as pointers and aliasing, complex control flow, and interprocedural propagation increase the difficulty of recovering dependency relations. These features also make slicing tools more prone to dependency omissions, redundant retention, and semantic distortion. Therefore, automated reliability validation for C program slicing tools is of significant research importance and practical value.

However, the testing of C program slicing tools still faces three major challenges. First, there is a lack of test oracles: slicing results are usually represented as statement sets, dependency relations, or intermediate representations, making it difficult to obtain low-cost and automatable correctness criteria. Second, deep semantic scenarios are insufficiently triggered: although traditional automated generation methods can expand the input space, their capability remains limited in targeted construction of scenarios involving complex aliasing relations, long-distance data flows, and fine-grained control dependencies. Third, heterogeneous result judgment is difficult: source-level and IR-level slicing tools differ significantly in input forms, dependency modeling granularity, and result representations, which further increases the difficulty of organizing a unified validation process and judging results.

To address the above challenges, this thesis applies the idea of metamorphic testing to the testing of C program slicing tools. Based on slicing semantics, four types of

domain-specific metamorphic relations are designed to transform the correctness judgment of slicing results into relation consistency checking, thereby alleviating the test oracle problem in slicing tool testing. To enhance the ability to trigger complex semantic scenarios, this thesis combines Csmith-based random generation with large-language-model-guided generation to construct test programs, so as to cover diverse program structures and complex dependency scenarios. At the automated detection level, this thesis designs an automated testing framework based on the above metamorphic relations. The framework organizes test program generation, slicing execution, result judgment, and defect organization for both source-level and LLVM IR-level slicing tools. In total, 43 reproducible issues are identified and organized, including 17 in Frama-C and 26 in dg; among the Frama-C issues, 13 have been confirmed by the developers. The experimental results show that the metamorphic-testing-based detection approach can be applied to the testing of C program slicing tools and can identify reproducible issues in automated defect detection, thereby validating the feasibility of the proposed method.

KEYWORDS: program slicing; metamorphic testing; static analysis; C programming language; large language models

目 录

目 录	V
插图目录	XI
表格目录	XIII
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 程序分析工具测试方法研究现状	2
1.2.2 蜕变测试在程序分析器验证中的应用	4
1.2.3 大语言模型在代码生成与软件测试中的应用	6
1.2.4 研究现状总结	7
1.2.5 研究聚焦问题	8
1.3 研究内容与主要贡献	9
1.4 论文结构	10
第二章 相关概念与技术基础	13
2.1 程序切片与依赖图基础	13
2.1.1 程序切片的定义与分类	13
2.1.2 切片准则	14
2.1.3 数据依赖与控制依赖	15
2.1.4 指针别名与数据依赖提取	15
2.1.5 程序依赖图与系统依赖图	16
2.2 典型 C 语言程序切片工具	17
2.2.1 Frama-C: 面向源码的分析平台	17

2.2.2	dg: 面向 LLVM Bitcode 的 IR 级切片器	18
2.2.3	LLVM IR 与调试信息	18
2.2.4	两类工具的实现路径差异	18
2.3	蜕变测试理论基础	19
2.3.1	测试预言问题与蜕变测试的基本思想	19
2.3.2	蜕变关系与任务实例化	20
2.3.3	蜕变测试流程	21
2.4	测试用例生成技术	21
2.4.1	基于 Csmith 的随机程序生成	22
2.4.2	基于大语言模型提示约束的定向生成	22
2.4.3	两类生成方式的特征差异	24
2.5	本章小结	24
第三章 面向 C 程序切片工具的蜕变关系设计		25
3.1	蜕变关系设计目标与原则	25
3.2	MR1: 面向语义保持的蜕变关系设计	26
3.2.1	设计动机	26
3.2.2	关系定义、理论依据与可执行判定原则	27
3.2.3	派生程序设计	27
3.2.4	C 语言示例	28
3.2.5	适用范围与局限	29
3.3	MR2 与 MR3 的共享构造流程	29
3.4	MR2: 面向无关数据流扰动的蜕变关系设计	31
3.4.1	设计动机	31
3.4.2	关系定义、理论依据与可执行判定原则	31
3.4.3	派生程序设计	32
3.4.4	C 语言示例	32
3.4.5	适用范围与局限	33
3.5	MR3: 面向无关控制流扰动的蜕变关系设计	33
3.5.1	设计动机	33

3.5.2	关系定义、理论依据与可执行判定原则	34
3.5.3	派生程序设计	34
3.5.4	实例化模式与 C 语言示例	35
3.5.5	适用范围与局限	36
3.6	MR4: 面向切片准则组合一致性的蜕变关系设计	36
3.6.1	设计动机	36
3.6.2	关系定义、理论依据与可执行判定原则	36
3.6.3	切片任务设计	37
3.6.4	C 语言示例	38
3.6.5	适用范围与局限	38
3.7	本章小结	39
第四章 面向 C 程序切片工具的蜕变测试框架		41
4.1	框架概述	41
4.1.1	整体框架简介	41
4.1.2	第一阶段: 种子程序准备	42
4.1.3	第二阶段: 蜕变任务实例化	44
4.1.4	第三阶段: 切片准则选择与切片执行	45
4.1.5	第四阶段: 分层结果整理、关系判定与状态输出	46
4.2	框架输出	46
4.3	本章小结	47
第五章 实验设计与结果分析		49
5.1	实验配置与统计口径	49
5.1.1	实验对象与配置	49
5.1.2	统计口径与复核方式	49
5.2	总体结果与工具差异	50
5.2.1	总体结果	50
5.2.2	问题类型分布	51
5.3	各蜕变关系揭示的问题类型	52
5.3.1	MR1 主要揭示的问题类型	52

5.3.2	MR2 主要揭示的问题类型	53
5.3.3	MR3 主要揭示的问题类型	53
5.3.4	MR4 主要揭示的问题类型	54
5.4	不同种子来源的贡献	54
5.5	代码覆盖率分析	56
5.5.1	Frama-C: 总体覆盖率与核心模块穿透	57
5.5.2	dg: 总体覆盖率与核心模块分布	58
5.6	典型缺陷根因与代表性案例	59
5.6.1	Frama-C: 依赖链恢复与源码导出自治性破坏	60
5.6.2	Frama-C: 函数指针与语言边界构造处理不一致	61
5.6.3	Frama-C: 多准则组合一致性问题	62
5.6.4	dg: 内存对象写入效应遗漏	63
5.6.5	dg: 回调与隐藏状态传播遗漏	64
5.6.6	dg: 非常规控制流与 IR/CFG 健壮性边界	66
5.7	有效性威胁	67
5.7.1	构念有效性	67
5.7.2	内部有效性	67
5.7.3	结论有效性	68
5.7.4	外部有效性	68
5.8	本章小结	69
第六章 总结与展望		71
6.1	研究总结	71
6.2	研究不足	72
6.3	未来展望	73
参考文献		75
致 谢		85
附录 A 问题项清单		87
A.1	第 5 章统一编号问题项清单	87

A.2	Frama-C 问题项的外部跟踪信息	89
A.3	dg 已提交问题的外部跟踪信息	90
附录 B LLM 提示词设计		91
B.1	LLM 种子程序的提示词模板与主题维度	91
B.1.1	基础提示词模板	91
B.1.2	关系导向附加约束	92
B.1.3	提示词主题维度	92
学位论文出版授权书		95

插图目录

1-1	论文章节结构与逻辑关系	10
2-1	程序切片示例	14
2-2	指针别名示例程序	16
2-3	程序切片示例程序对应的程序依赖图	16
2-4	不同实现路径的 C 程序切片流程示意	19
4-1	面向 C 程序切片工具的蜕变测试框架	41
5-1	两种切片工具不同类型问题项数量分布	52
5-2	<i>Frama-C</i> 四类蜕变关系测试在四组核心模块上的覆盖率对比	58
5-3	<i>dg</i> 四类蜕变关系测试的行覆盖率与分支覆盖率对比	59

表格目录

3-1	四类蜕变关系的设计目标、可执行判定原则与主要风险	26
3-2	MR2 与 MR3 的主要构造方式及示例模板	30
4-1	种子程序准备阶段的四类检查	42
4-2	Csmith 与大语言模型在种子程序构造中的作用	43
4-3	第二阶段蜕变任务实例化的输入项与输出项	44
4-4	切片准则确定的三类策略	45
4-5	异常案例输出的主要记录项	47
5-1	实验环境与工具配置	49
5-2	两种工具的问题状态分布	50
5-3	按主归因关系划分的问题项数量	52
5-4	四类关系揭示的问题类型与对应讨论位置	55
5-5	不同种子来源揭示的问题项数量	55
5-6	Frama-C 核心模块最高覆盖率	57
5-7	dg 核心模块最高覆盖率	58
A-1	第 5 章统一编号问题项及其主归因关系与状态	87
A-2	Frama-C 已确认问题项的公开 issue 跟踪信息	89
A-3	Frama-C 已提交问题项的补充档案信息	89
A-4	dg 已提交问题的 GitHub issue 范围	90
A-5	dg 已提交问题对应的统一编号问题项	90
B-1	面向四类蜕变关系的 LLM 种子程序附加约束	92
B-2	LLM 种子程序提示词的主题维度及其对应分析环节	93

第一章 绪论

1.1 研究背景与意义

程序切片（Program Slicing）是程序分析与软件工程中的基础技术^[1]。其基本思想是在给定切片准则的条件下，从程序中抽取所有可能影响目标变量取值或目标语句执行行为的语句集合。通过在保留相关语义依赖的前提下缩小程序规模，程序切片能够帮助分析者聚焦于与目标行为直接相关的代码区域，因此被广泛应用于程序理解、软件维护、故障定位、程序比较和安全分析等任务^[2-6]。

随着应用需求扩展，程序切片逐步从早期静态切片发展出多种变体。静态切片面向程序所有可能执行路径进行分析，适用于程序理解和整体分析；但在具体执行场景中，静态切片结果往往包含较多与当前执行无关的语句，可能影响分析精度。为适应不同分析目标在精度、表达能力和适用场景上的差异，研究者进一步提出了动态切片、混合切片和非保形切片（amorphous slicing）等方法^[7-14]。围绕程序依赖表示、形式语义和算法实现的研究也持续推进，使程序切片逐渐从概念性分析方法发展为支撑真实软件工程任务的重要技术^[15-19]。

在安全分析场景中，程序切片具有基础性作用。污点传播路径追踪、敏感变量影响范围判定、漏洞触发条件回溯等任务，都依赖切片对程序依赖关系的有效恢复；相关研究也利用程序切片提取漏洞相关代码片段，并结合模型进行漏洞检测^[6,20]。若切片结果遗漏关键依赖，后续分析可能产生漏报；若切片结果保留大量无关代码，分析人员又会面对过多噪声。因此，切片工具的可靠性直接影响漏洞定位、代码审计和安全分析结果的可信度。

这一问题在 C 语言场景中更加突出。大量底层系统软件、嵌入式软件和安全关键基础设施仍以 C 语言实现，而 C 语言也是程序分析中难度较高的语言之一。指针与多级解引用、数组与指针的等价性、显式内存管理、函数指针、结构体字段别名、宏展开和复杂控制流等特性，使程序中的数据依赖与控制依赖难以被精确恢复。真实工程代码还常伴随更深的过程调用、更复杂的跨函数传播和更

频繁的条件编译，使切片工具在精度、可扩展性和运行开销之间必须进行折中。

工业级切片工具通常采用保守近似、简化建模或启发式处理策略，以保证在大规模代码库上的可用性。这类设计提高了工程适配能力，但也可能在实现层面引入依赖遗漏、冗余保留或结果导出偏差。程序切片工具并非普通软件组件，而是上层程序分析系统的重要基础。一旦切片结果不可靠，下游分析链条的解释力与可信度也会下降。

然而，面向切片工具本身的系统化验证仍然不足^[21-22]。与普通程序输出不同，切片结果通常表现为非定长语句集合、依赖图或中间表示片段，难以通过单一数值或固定答案直接判断正确性。这类困难在软件测试领域被称为测试预言问题（Oracle Problem）^[23]。对于复杂 C 程序和切片准则，精确判断某条语句是否应被保留，需要同时分析数据依赖、控制依赖、别名关系和过程间传播路径。人工构造标准答案成本高，自动生成绝对正确答案同样困难，因而限制了切片工具测试的规模化扩展。

除测试预言缺失外，构造能够触发切片工具深层逻辑缺陷的测试程序也具有难度。常规随机生成方法可以扩大输入空间，但未必能稳定覆盖指针别名、回调传播、库函数副作用和非常规控制流等高风险语义场景。不同实现层级的切片工具还存在输入形式、依赖建模粒度和结果表示差异，使统一比较和问题归因更加困难。

综上，C 程序切片工具的可靠性验证面临判定依据匮乏、深层语义场景构造困难和工具间结果异构等问题。如何在不依赖唯一标准切片答案的前提下，系统化、自动化地对 C 程序切片工具开展缺陷检测与可靠性评估，是程序分析工具质量保障中的重要问题。围绕这一问题开展研究，对于提升底层程序分析基础设施和安全审计工具的可信度具有理论意义和工程价值。

1.2 国内外研究现状

1.2.1 程序分析工具测试方法研究现状

针对编译器和程序分析工具的测试，学术界与工业界已经形成了若干常用技术路线。基础方法是人工构造单元测试与回归测试集，即由开发者围绕历史缺陷、边界场景和语言特性手动编写测试程序。这类方法实现简单、便于定向回

归，但覆盖面高度依赖人工经验，难以充分覆盖复杂语言构造及其组合空间。为提高测试效率，研究者进一步提出了基准集驱动、差分测试、随机测试、翻译验证和形式化验证等方法。其中，差分测试通过比较多个实现对同一输入的行为差异发现潜在错误^[24]；随机测试或模糊测试通过大规模生成输入触发隐藏缺陷；翻译验证与形式化验证则尝试从语义层面检查实现与规范之间的一致性。

在编译器测试领域，Csmith 的成功具有代表性。该工具能够生成语法正确且无未定义行为的 C 程序，并据此在 GCC、LLVM 等系统中发现大量真实缺陷；YARPGen 则进一步说明，围绕表达能力与优化触发策略进行设计的随机生成器能够在工业编译器中持续发现高价值问题^[25-26]。随后，EMI、Alive2 等工作又分别从语义保持变换和翻译验证角度拓展了复杂软件基础设施的验证能力^[27-28]。近年研究还开始将真实代码片段注入编译器测试过程，以提高测试程序与工程代码之间的接近程度^[29]。

面向程序分析器本身的验证也在持续推进。例如，已有工作针对 C 静态分析器开展形式化验证^[30]，针对 LLVM 数据流分析开展精度与健全性测试^[31]。围绕静态分析器测试，StatFier 通过语义保持程序变换测试静态分析器^[21]；He 等通过构造自动化判定依据挖掘静态分析器缺陷^[22]；也有研究围绕安全代码审查实践，对静态分析工具开展经验研究^[32]。这些研究表明，程序分析工具测试除关注崩溃、超时等鲁棒性问题外，也逐渐关注分析结果的健全性、精度和工程可用性。

程序切片工具属于程序分析工具的重要类型，但其测试问题具有更强的结果判定特殊性。切片结果通常表现为语句集合、依赖关系、依赖闭包或中间表示片段，其正确性依赖数据依赖、控制依赖、别名关系和过程间传播等语义信息。已有研究较早关注程序切片器测试问题。例如，Pérez 等提出自动化测试程序切片器的方法，通过构造候选切片并检查切片性质来发现切片器问题^[33]；ORBS 则通过观察程序行为验证潜在切片的正确性，为切片验证提供了不同于传统静态比对的思路^[34]。这些工作说明，程序切片工具测试不能简单依赖固定标准输出，而需要结合切片性质设计专门的判定方式。

除直接面向切片器测试的工作外，部分研究从切片结果质量以及切片在验证任务中的作用出发，对程序切片进行了进一步评估。Stiévenart 等提出准静态可执行切片（quasi-static executable slices），关注传统静态切片结果与可执行切片

之间的差异，并将其与基于观察的切片（**observation-based slicing**）和静态切片进行实证比较^[35]。Chai 等研究了程序切片在软件验证前处理中的作用，分析不同切片技术与验证器组合对验证效率和验证结果的影响^[36]。这类工作表明，切片结果的质量、可执行性及其对验证任务的影响仍是相关研究关注的问题。不过，它们更多面向切片方法评估或切片辅助验证，尚未直接解决真实 C 程序切片工具的自动化缺陷检测问题。

总体来看，复杂软件基础设施测试已经在编译器、静态分析器和程序验证工具等方向形成了较多方法积累。然而，这些方法迁移到程序切片工具时仍会遇到明显障碍。首先，差分测试依赖“多个实现输出可直接比较”这一前提，但不同切片工具在内部建模方式、输出粒度和保守策略上往往差异显著，即便面对同一程序和相同切片准则，也可能给出形式不同但语义上近似的结果，直接比较容易引入伪差异。其次，传统随机测试更容易触发解析器崩溃等鲁棒性问题，而切片工具中的关键错误往往位于依赖分析、别名传播和过程间图构建等逻辑模块，只有具备较强语义密度和结构约束的测试程序才更可能触发此类缺陷。再次，形式化验证虽然具有较强的理论保证，但建模成本高、扩展性有限，难以直接覆盖真实工具中的大量工程化细节。因此，面向程序切片工具的测试研究仍相对有限，尤其缺少面向 C 语言真实切片工具的可自动化、可复核缺陷检测流程。

1.2.2 蜕变测试在程序分析器验证中的应用

蜕变测试是解决测试预言缺失问题的重要方法。这里所说的测试预言，是指用于判断程序输出或分析结果是否正确的依据。蜕变测试并不要求测试者事先知道某个输入对应的绝对正确输出，而是先建立输入变换与输出约束之间的蜕变关系（**Metamorphic Relations, MRs**），再通过检查待测系统在原始输入与跟随输入上的结果是否满足该关系，间接判断系统是否可能存在缺陷^[37-39]。随着相关研究的发展，蜕变测试已经从面向特定科学计算问题的测试方法，扩展到机器学习、编译器、程序分析工具和软件基础设施等结果难以直接判定的系统；围绕其挑战、适用场景和工程实践的总结也较为成熟^[40-42]。

在程序分析相关研究中，蜕变测试尤其适合那些“输出难以直接判真，但输入变换前后的语义约束相对稳定”的对象。除编译器中的 EMI 之外，研究者还将其用于分类器和机器学习系统^[43-45]、交互式调试器^[46]以及编译器并发支持等

软件基础设施^[47]。这些研究表明，当标准答案难以穷举时，围绕输入变换与输出约束建立间接判定机制，通常比逐点构造标准结果更加可行。

蜕变测试也被用于程序分析基础设施的验证。例如，Mansur 等提出面向 Datalog 引擎的依赖感知蜕变测试方法^[48]。Datalog 被广泛用于程序分析任务，其引擎结果的正确性会影响上层分析结果的可靠性。该工作通过构造满足依赖约束的查询变换，检查不同查询结果之间是否满足预期关系，从而发现 Datalog 引擎中的错误。这类研究说明，蜕变测试不仅适用于普通程序输出，也可以用于具有结构化语义约束的程序分析基础设施。

与程序切片更接近的工作也开始出现。Hirsch 和 Hofer 在动态切片与相关切片研究中提出 *pruned slicing*，用于缩短动态切片和相关切片的长度，并在原型实现测试中使用蜕变测试处理测试预言问题^[49]。该工作表明，切片结果验证与蜕变测试之间具有一定契合性。不过，其研究对象主要是特定语言环境下的动态/相关切片原型，关注点在于切片长度缩减和原型实现评估，并不是面向真实 C 静态切片工具的系统化缺陷检测。

与此同时，蜕变关系本身的构造、测试组生成与执行优化也从经验驱动逐步走向系统化。METRIC 等研究尝试基于类别选择框架辅助识别蜕变关系^[50]；基于因果图的关系构造将程序中的因果依赖显式纳入 MR 设计过程^[51]；其他相关工作则从更广的角度总结了蜕变关系生成的研究现状与未来方向^[52]。在大语言模型兴起后，已有工作开始探索借助 ChatGPT 自动生成候选蜕变关系的经验效果^[53]。这些工作说明，蜕变测试的研究重点已经不局限于具体应用场景，还包括关系识别、关系生成、测试组构造和执行优化等支撑技术^[50-54]。

尽管已有研究为测试预言缺失问题提供了重要方法基础，蜕变测试在程序切片工具验证中的系统化应用仍然不足。现有应用更多集中于编译器、机器学习系统、Datalog 引擎和一般软件基础设施；直接面向 C 语言静态切片工具的领域专用蜕变关系设计仍较少。对于 C 程序切片工具而言，常见失效往往与多级指针、别名传播、跨函数依赖、控制依赖建模以及不同实现层级下的结果表示差异相关，仅依赖通用的“等价输入—等价输出”思想难以充分揭示这类深层逻辑偏差。因此，如何将切片语义中的依赖保持、非干扰性和多准则一致性等性质转化为可执行、可复核的蜕变关系，并落实到真实工具的自动化验证流程中，仍是一个有待解决的问题。

1.2.3 大语言模型在代码生成与软件测试中的应用

近年来,大语言模型在代码生成、程序理解与软件测试领域受到广泛关注。已有研究将其应用于单元测试生成、测试输入生成、模糊测试增强、编译器测试以及静态分析辅助等任务^[55-59]。例如,Fuzz4All 利用大语言模型生成和变异多种语言或系统的测试输入,以支持通用化模糊测试^[56];WhiteFox 将大语言模型引入白盒编译器测试,用于增强面向编译器缺陷的测试程序构造能力^[57];ChatUniTest 通过自适应上下文构造与生成、验证、修复流程辅助单元测试生成^[60];也有研究利用大语言模型生成或补充函数摘要,以改进静态分析结果并降低误报^[58]。这些工作表明,大语言模型已经被用于软件测试和程序分析流程中的多个环节,包括测试输入构造、程序变换、测试用例生成和静态分析辅助等。

从方法基础上看,大语言模型在软件测试和程序分析中的作用主要体现在以下几个方面。首先,大语言模型具有较强的代码语义理解与模式组织能力,能够根据已有代码、自然语言描述或结构化约束生成语法上相对完整、语义上相对连贯的程序片段。其次,大语言模型能够围绕特定目标生成包含特定语言特性、控制结构或数据流模式的测试用例,从而补充传统随机生成方法在定向构造方面的不足。再次,大语言模型可以辅助程序补全、程序改写、测试任务生成、结果解释和缺陷报告整理等任务,为自动化测试流程提供更灵活的语义支持^[55-56,58]。因此,大语言模型并不只是简单的代码生成工具,而是可以作为软件测试与程序分析流程中的通用增强组件。

与程序切片相关的研究也开始关注大语言模型的潜在作用。Shahandashti 等考察了多种大语言模型在静态切片和动态切片任务上的表现,说明大语言模型具备一定的程序依赖理解和切片任务处理能力^[61]。这类研究表明,大语言模型可以为程序切片相关任务提供新的技术路径,例如辅助理解程序依赖关系、生成切片相关样例、组织切片准则或支持结果解释。但由于程序切片涉及精确的数据依赖、控制依赖和过程间传播分析,LLM 的输出通常仍需要结合编译检查、规则约束、关系判定或人工复核加以约束和确认。

将大语言模型引入程序切片工具测试时,其价值不仅体现在生成程序样例,还体现在对测试任务的组织和语义场景的构造上。程序切片工具的输出通常是语句集合、依赖关系或中间表示片段,而不是单一数值或简单文本结果,因此测

试过程需要同时考虑测试程序是否包含明确的依赖结构、切片准则是否能够触发目标语义场景，以及不同切片结果之间是否满足预期关系。对于 C 程序切片工具而言，指针别名、多级解引用、函数指针、回调、跨函数传播、库函数副作用和复杂控制依赖等结构都会影响切片结果。大语言模型能够根据这些目标约束生成或改写测试程序，也可以辅助整理异常现象、分析结果差异和形成复核材料，从而增强自动化测试流程对复杂语义场景的覆盖能力。

同时，大语言模型在程序切片工具测试中的应用仍需要与传统程序分析和测试判定机制结合。一方面，模型生成的 C 程序可能存在编译失败、约束满足不足或与目标语义场景不一致等问题，需要通过编译检查、运行检查和规则过滤进行筛选；另一方面，切片结果的正确性仍需要依赖明确的判定原则或可复核的关系约束。由此可见，大语言模型更适合作为程序切片工具测试流程中的增强手段，与随机程序生成、蜕变关系判定和结果复核机制共同发挥作用，而难以单独构成完整的切片工具验证方案。

1.2.4 研究现状总结

综合已有研究，程序切片、程序分析工具测试、蜕变测试和大语言模型辅助测试已经为 C 程序切片工具验证提供了重要基础。已有工作分别从切片算法与应用、程序分析器测试、蜕变关系构造以及测试输入生成等方面提供了方法支撑。然而，当研究对象转向真实 C 程序切片工具时，现有方法仍难以直接满足系统化、自动化和可复核缺陷检测的需求，主要面临以下三方面挑战。

首先，程序切片结果通常表现为语句集合、依赖关系或中间表示片段，难以直接获得低成本、可自动化的标准答案。因此，程序切片工具测试的第一个挑战是：**测试预言缺失：切片结果通常表现为语句集合、依赖关系或中间表示，缺乏低成本、可自动化的正确性判定依据。**

其次，C 程序切片结果高度依赖指针别名、数据流传播和控制依赖等语义结构，而传统测试输入生成方法难以稳定覆盖这类场景。因此，程序切片工具测试的第二个挑战是：**深层语义场景触发不足：传统自动生成方法虽然能够扩大输入空间，但对复杂别名关系、长距离数据流和细粒度控制依赖等场景的定向构造能力仍然有限。**

最后，源码级与 IR 级切片工具在输入形式、依赖建模粒度和结果表示方式

上存在差异，增加了统一组织测试与判定的难度。因此，程序切片工具测试的第三个挑战是：**异构结果判定困难**：源码级与 IR 级切片工具在输入形式、依赖建模粒度和结果表示方式上存在明显差异，进一步增加了统一组织验证流程与结果判定的难度。

针对上述挑战，本文将蜕变测试引入 C 程序切片工具验证场景，结合领域专用蜕变关系、双轨测试输入构造和分层结果整理机制，构建面向真实 C 程序切片工具的自动化缺陷检测方法。

针对 C 程序切片工具验证中的测试预言缺失、语义场景触发不足和异构结果判定困难等问题，本文选取源码级切片工具 Frama-C 与 LLVM IR 级切片工具 dg 作为代表性研究对象^[62-63]，将蜕变测试思想引入 C 程序切片工具的自动化验证过程。本文围绕蜕变关系设计、测试输入构造和结果判定机制展开研究，并通过实证评估分析所提方法在真实切片工具上的适用性。

1.2.5 研究聚焦问题

为使方法设计、实验分析与结论归纳形成闭环，本文将实证研究部分聚焦于以下四个研究问题：

1. RQ1：所提方法能否在真实 C 语言程序切片工具上发现可复现的功能性失效？

该问题对应本文方法的基本有效性，关注关系驱动的自动化验证流程能否在源码级与 IR 级两类真实工具上持续产生具有分析价值的异常信号，并经过复核确认其为可复现问题。

2. RQ2：不同蜕变关系在缺陷揭示中的作用是否存在差异？

该问题关注不同类型蜕变关系在问题发现与偏差定位中的分工，考察它们分别更容易揭示何种类型的切片偏差，以及是否能够对数据流、控制流和多准则一致性层面的缺陷形成互补。

3. RQ3：大语言模型引导生成与传统随机生成在不同工具和缺陷类型上分别发挥何种作用？

该问题关注 Csmith 随机生成与大语言模型引导生成在不同工具、不同缺陷类型和不同语义结构上的作用差异，以及二者是否能够形成互补。

4. RQ4: 所提方法是否能够更充分地进入切片工具的关键实现路径, 并揭示不同实现层级上的薄弱点?

该问题关注测试样例对依赖分析、结果导出和 IR 处理等关键实现区域的触达情况, 考察测试是否真正进入与切片语义相关的核心路径, 并在这些区域揭示具有解释力的问题模式。

1.3 研究内容与主要贡献

围绕上述研究问题, 本文从蜕变关系设计、测试输入构造以及自动化验证框架三个方面展开研究。主要研究内容与贡献可归纳如下:

1. **提出了面向 C 程序切片工具的领域专用蜕变关系体系。**针对切片结果难以直接判定正确性的问题, 本文围绕程序切片中的依赖保持、数据非干扰、控制非干扰和多准则组合一致性等语义性质, 设计了四类领域专用蜕变关系。该体系将“切片结果是否绝对正确”的判定问题转化为“相关切片结果之间是否满足关系一致性”的检查问题, 为 C 程序切片工具测试提供了可执行的间接测试预言。
2. **设计了结合 Csmith 与大语言模型的双轨测试输入构造方法。**针对切片敏感语义场景触发不足的问题, 本文结合随机生成与目标化生成两类方式构造测试程序。一方面, 利用 Csmith 生成语法正确且无未定义行为的随机 C 程序, 以覆盖常规程序结构和工具前端处理路径; 另一方面, 利用大语言模型在提示约束下生成包含指针别名、函数指针、回调传播、全局状态、库函数副作用和复杂控制流等特征的测试程序, 以增强对切片核心逻辑的定向探测能力。
3. **构建了面向真实 C 程序切片工具的自动化验证框架并开展实证评估。**针对源码级工具与 LLVM IR 级工具在输入形式、结果表示和依赖建模粒度上的差异, 本文设计了分层结果整理与关系判定机制, 并将测试程序生成、切片执行、结果提取、关系检查和异常整理组织为自动化测试流程。基于该框架, 本文在 Frama-C 和 dg 两类代表性切片工具上开展实验评估, 从问题发现能力、不同蜕变关系作用、不同输入来源贡献以及工具关键实现路径覆盖等方面验证了所提方法的可行性。

上述工作共同构成了本文面向 C 程序切片工具的自动化测试方法：领域专用蜕变关系用于缓解测试预言缺失问题，双轨测试输入构造用于增强切片敏感语义场景覆盖，分层验证框架用于支持不同实现层级切片工具上的自动化执行与结果判定。

1.4 论文结构

本文按照“问题提出—基础支撑—方法设计—方法框架—实验分析—总结展望”的主线组织全文，共分为六章。图 1-1 给出了各章内容及其逻辑关系。

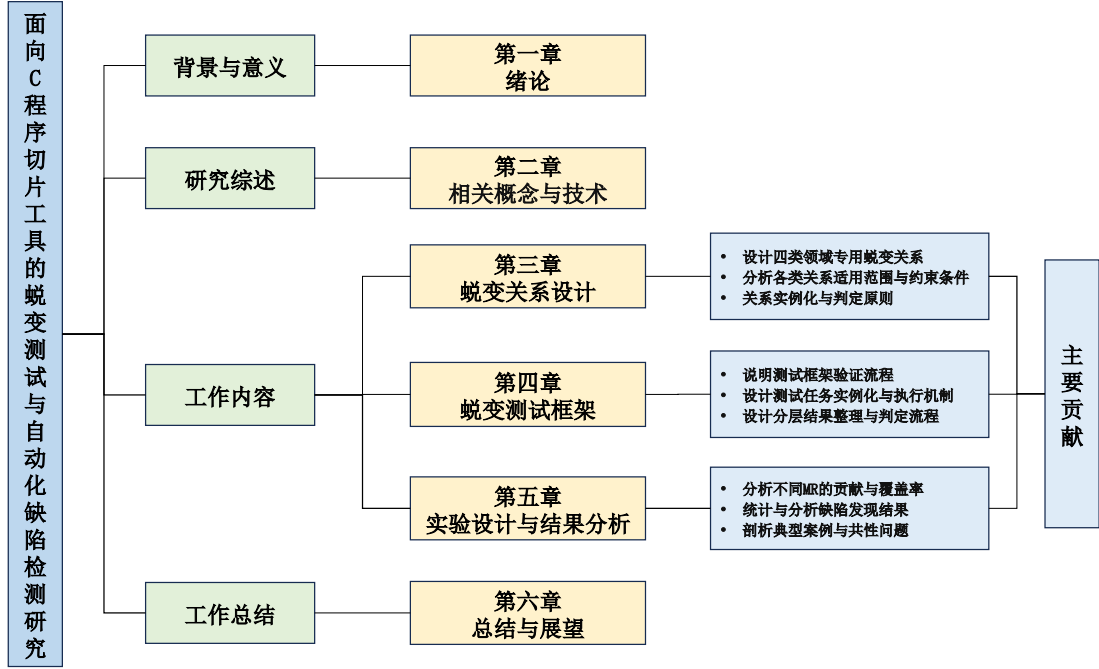


图 1-1 论文章节结构与逻辑关系

第一章为绪论，主要介绍研究背景与意义、国内外研究现状、研究问题、主要研究内容与贡献，并给出全文结构安排。

第二章为相关概念与技术基础，介绍程序切片、依赖分析、蜕变测试、典型切片工具以及测试输入生成技术，为后续研究提供基础。

第三章设计面向 C 程序切片工具的领域专用蜕变关系，围绕语义保持、数据非干扰、控制非干扰和多准则一致性给出关系定义、判定原则与适用边界。

第四章提出面向 C 程序切片工具的蜕变测试框架，说明种子程序准备、任务实例化、切片执行、结果整理和异常输出的整体流程。

第五章开展实验设计与结果分析，说明实验配置与统计口径，分析 Frama-C

和 **dg** 上发现的问题项、不同蜕变关系与输入来源的作用，以及典型缺陷案例。

第六章总结全文工作，围绕研究问题归纳主要结论，分析方法局限，并讨论后续改进方向。

第二章 相关概念与技术基础

本章介绍后续研究所涉及的相关概念与技术基础，内容包括程序切片与依赖图、数据依赖分析及指针别名、典型 C 语言程序切片工具、蜕变测试理论基础以及测试用例生成技术。上述内容为后续验证框架构建、蜕变关系设计、分层结果整理与关系判定提供必要的概念与技术支撑。

2.1 程序切片与依赖图基础

程序切片（Program Slicing）是一类围绕给定分析目标抽取语义相关程序部分的程序分析技术^[1,15-17]。从本质上看，程序切片并不是对程序文本的简单删减，而是依据程序中的数据依赖和控制依赖，围绕特定切片准则保留语义相关的程序部分^[64-65]。因此，要准确理解程序切片，需要进一步说明其定义、分类、切片准则以及依赖关系的表示方式。

2.1.1 程序切片的定义与分类

程序切片通常是指：在给定切片准则的条件下，从原程序中提取出所有可能影响该准则所指定计算结果的语句集合，或者提取出所有可能受到该准则所指定语句影响的语句集合^[1,16,66]。这里的“影响”主要体现在数据传递和控制结构两个方面，因此程序切片本质上是一种依赖分析过程。

根据分析方向的不同，程序切片通常可分为前向切片和后向切片两类^[15,17]。前向切片（Forward Slicing）关注某一程序点可能对后续程序行为造成的影响范围；后向切片（Backward Slicing）则关注某一程序点的值或行为受到哪些前置语句影响。相较而言，后向切片更常用于错误定位和程序理解，因为它能够帮助研究者追溯某一结果的来源。

除按方向分类外，程序切片还可以按分析方式分为静态切片和动态切片。静态切片在不依赖具体输入的情况下分析程序所有可能执行路径，因此适用范围

较广，但结果通常较大；动态切片则基于特定输入和实际执行轨迹进行分析，其结果通常更加精确，但依赖于给定执行场景^[7-8,10-11]。

2.1.2 切片准则

切片准则（Slicing Criterion）是程序切片中的核心概念，用于明确切片分析的目标。一般而言，一个切片准则由程序位置和相关变量共同构成，用来表示研究者希望关注的程序点及其语义信息^[1,15,66]。

其中，程序位置通常表示某一条语句、某个程序点或某个基本块；相关变量则表示在该位置上需要被关注的变量集合。例如，若研究者希望分析某一输出语句中变量值的来源，则可以将该输出语句及相应变量作为切片准则，并据此构造后向切片。

切片准则的设置直接影响切片结果的范围。若切片准则较为宽泛，则保留下来的语句数量通常较多；若切片准则较为精确，则切片结果通常更加集中。因此，切片准则既决定了程序切片的分析目标，也决定了切片结果的粒度^[15,17]。

例如，考虑如下程序片段。若将“第 9 行变量 z ”作为切片准则，则后向切片需要保留所有可能影响该输出位置上 z 值的语句，包括第 1 行、第 3 行以及第 5 行至第 9 行；其中，第 2 行语句 $t = 100$ ；与该准则无关，因此不属于切片结果。图 2-1 给出了相应示例。

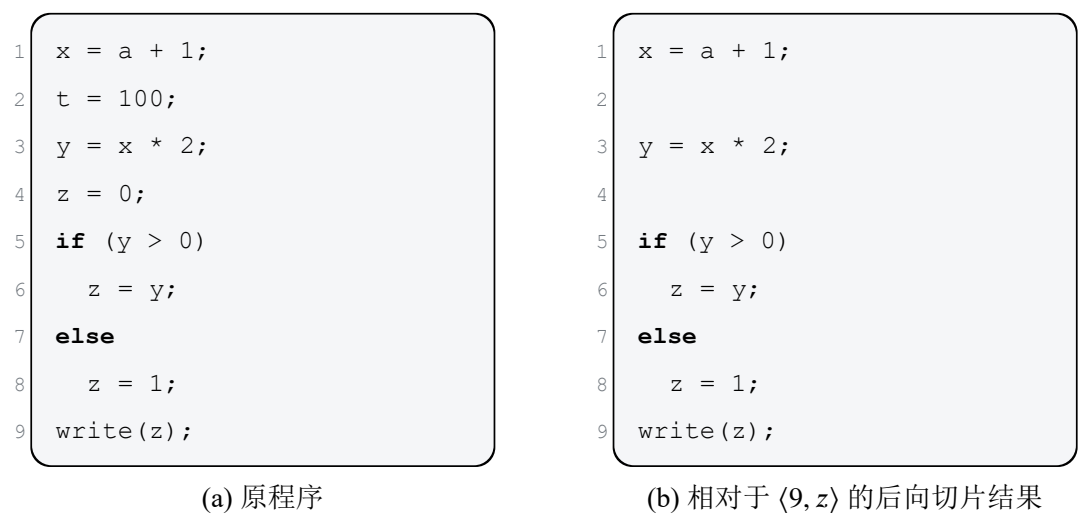


图 2-1 程序切片示例

第 4 行对 z 的初始化在后续两个分支中都会被重新赋值覆盖，因此在该示

例的理论后向切片中通常可以不被保留。但在工程实践中，部分采用保守近似策略的静态切片工具仍可能将其纳入结果。

2.1.3 数据依赖与控制依赖

程序切片的构造基础在于程序依赖关系。通常而言，程序依赖主要包括数据依赖（Data Dependence）和控制依赖（Control Dependence）两类^[5,64]。

数据依赖描述的是程序中值的传递关系。当某条语句定义了一个变量，而另一条语句随后使用了该变量，且在两者之间不存在对该变量的重新定义时，就认为这两条语句之间存在数据依赖关系^[64-65]。数据依赖体现了程序状态在不同语句之间的传播过程，是分析变量来源和理解计算结果形成机制的重要依据。

控制依赖描述的是程序中执行路径的约束关系。当某条语句的执行与否取决于某个条件判断或分支结构时，就认为该语句受相应判断语句的控制依赖^[64]。控制依赖反映了程序结构对执行行为的约束，是分析程序执行条件和路径选择的重要依据。

在程序切片中，仅考虑数据依赖通常是不够的，因为某条语句即使参与了变量定义，也只有在相应控制条件满足时才会真正执行。因此，程序切片通常需要同时考虑数据依赖和控制依赖，以保证切片结果能够反映原程序的语义上下文。

2.1.4 指针别名与数据依赖提取

在 C 语言中，数据依赖的识别往往不仅取决于显式变量名，还取决于指针所指向的内存对象。若某条语句通过指针写入某个内存位置，而另一条语句通过相同或可能别名的指针读取该位置，则两者之间可能存在数据依赖。因此，精确的数据依赖提取通常依赖于指针分析或别名分析（Alias Analysis / Points-to Analysis）对指针指向关系的刻画。

然而，别名分析本身往往难以做到完全精确。若分析结果过于保守，将多个本不相同的内存位置归入同一别名集合，则依赖分析可能引入额外的数据依赖边，进而导致切片结果保留冗余语句；反之，若别名关系建模不充分，则可能遗漏真实依赖，造成切片欠近似。对于 C 语言程序切片而言，指针别名分析的精度因此直接影响切片结果的准确性与可解释性。

```

1  int x = 0;
2  int *p = &x;
3  int *q = &x;
4  *p = 1;
5  t = *q;

```

图 2-2 指针别名示例程序

以图 2-2 为例，语句第 (4) 行 $*p = 1;$ 与第 (5) 行 $t = *q;$ 之间是否存在数据依赖，取决于分析对指针别名关系的刻画。在该示例中， p 与 q 都指向变量 x ，因此第 (4) 行对内存对象的写入会影响第 (5) 行对同一对象的读取，二者之间真实存在数据依赖。

若分析未能识别这一别名关系，则应有的数据依赖边将不会建立，进而可能使后向切片遗漏本应保留的定义语句，表现为欠保留。指针别名分析的精度会直接影响数据依赖提取结果，并进一步影响切片结果的准确性与可解释性。

2.1.5 程序依赖图与系统依赖图

为了统一表示程序中的依赖关系，研究中通常引入程序依赖图（Program Dependence Graph, PDG）等图结构表示方法^[64,67]。程序依赖图是一种将程序语句及其依赖关系显式建模的有向图，其中节点通常表示程序中的语句或谓词，边则表示节点之间的数据依赖或控制依赖关系。图 2-3 给出了程序切片示例程序对应的程序依赖图。

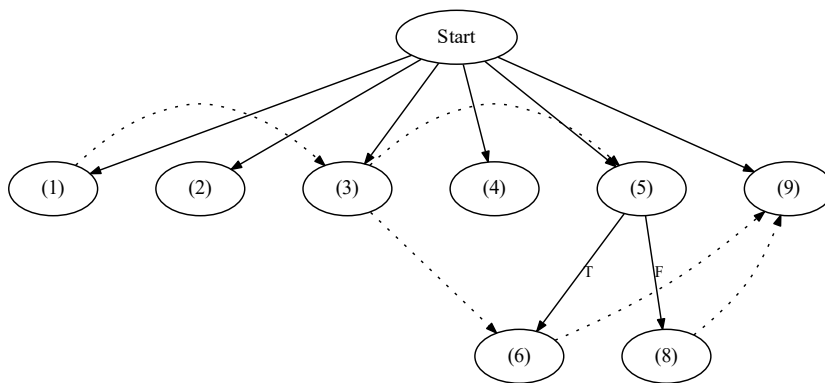


图 2-3 程序切片示例程序对应的程序依赖图

在图 2-3 中，节点编号与前述程序切片示例中的语句行号相对应。图中的依赖边用于显式表示语句之间的语义关系：其中，控制依赖边连接条件判断节点与受其控制的语句节点，数据依赖边连接变量定义节点与变量使用节点。借助这种图结构，原本隐含在程序文本中的语义关系得以显式表示，从而支持切片、优化和程序理解等分析任务^[5,64]。

对于包含过程调用的程序，单一过程内部的程序依赖图通常不足以完整反映跨过程影响关系。为此，研究中进一步提出了系统依赖图（System Dependence Graph, SDG），在程序依赖图的基础上显式表示过程调用、参数传递和返回值传播等过程间依赖关系^[5,65]。因此，SDG 可以视为支持过程间程序切片的重要扩展表示。

2.2 典型 C 语言程序切片工具

随着程序分析技术的发展，程序切片已经从理论研究逐步走向工具化实践。程序切片工具通常通过解析程序、构建控制流与依赖关系、接收切片准则并输出切片结果等步骤完成自动化切片分析^[15,68]。从分析层级上看，切片工具既可以直接面向源代码，也可以面向编译器中间表示。前者更接近原始程序结构，后者更适合在统一中间表示上开展底层依赖建模与分析实现^[34,62-63]。

2.2.1 Frama-C：面向源码的分析平台

Frama-C 是面向 C 源代码的程序分析平台，支持在源码层面开展静态分析、程序验证以及相关程序变换^[62,69]。对于程序切片任务而言，Frama-C 的分析对象直接是 C 源程序，切片结果也更贴近源码结构，因此便于从源代码层面对保留语句、控制结构和依赖传播进行观察与解释。

源码级切片具有较强的可解释性。研究者可以直接观察被保留的语句、条件结构以及函数调用在源码中的位置，从而判断切片结果是否与语义预期一致。但这类工具也需要面对源语言本身带来的复杂性，例如预处理、宏展开、库函数语义建模以及复杂语法构造对依赖提取的影响。

2.2.2 dg: 面向 LLVM Bitcode 的 IR 级切片器

与 Frama-C 不同, dg 是基于 LLVM 中间表示开展依赖分析与程序切片的工具, 其分析对象通常是由 C 程序编译得到的 LLVM IR 或 bitcode^[63]。相较于直接面向源代码的工具, 基于中间表示的切片工具更容易在统一的底层表示上实现依赖建模、别名分析和过程间传播分析, 因此在工程实现上具有较强的一致性与可扩展性。

由于 dg 的切片结果直接产生于 LLVM IR / bitcode 层, 其输出形式也更接近中间表示, 而非源码层表示。与源码级切片工具相比, 这类工具在依赖构造、结果组织与后续解释方式上通常具有不同特点。

2.2.3 LLVM IR 与调试信息

LLVM IR 是 LLVM 编译基础设施中的一种中间表示形式, 位于高级语言源代码与底层机器代码之间。相较于直接处理源代码, 在 LLVM IR 上开展分析能够获得更统一的程序表示形式, 从而便于实现底层依赖建模、别名分析和过程间传播分析。因此, 许多面向底层语义的程序分析工具会选择在 LLVM IR 层面实现其核心分析逻辑。

在这一表示层中, 调试信息可以为 IR 指令、基本块与源码位置之间建立辅助对应关系, 从而支持源码级理解、定位与调试^[70]。因此, 调试信息承担的是辅助解释作用, 而不是切片结果本身的表示形式。

2.2.4 两类工具的实现路径差异

综合来看, Frama-C 与 dg 分别代表源码级与中间表示级程序切片两条典型实现路径。前者更接近源语言结构, 便于从程序文本层面理解结果; 后者更依赖统一 IR 上的底层依赖建模, 便于在编译基础设施之上开展分析实现。二者在输入形式、依赖建模粒度和结果表示方式上的差异, 也构成了源码级与 IR 级切片工具在实践中的主要区别。

为更直观地说明源码级与 IR 级程序切片工具在实现路径上的差异, 图 2-4 给出了 Frama-C 与 dg 的典型切片流程。

图 2-4 表明, Frama-C 的切片计算直接面向 C 源程序展开, 其前端解析、依

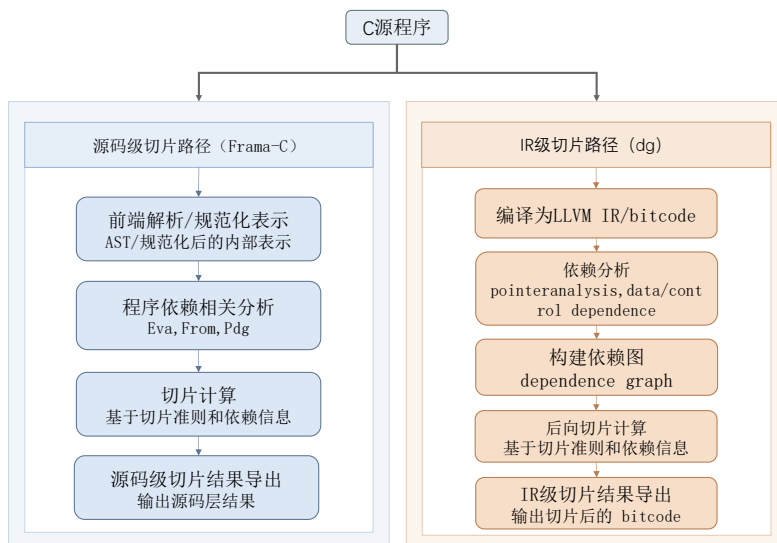


图 2-4 不同实现路径的 C 程序切片流程示意

赖相关分析和切片结果导出均更接近源码层表达；相比之下，dg 的切片流程首先在 LLVM IR / bitcode 层完成，依赖分析、依赖图构建和后向切片均围绕中间表示展开，并最终导出切片后的 bitcode。

2.3 蜕变测试理论基础

蜕变测试（Metamorphic Testing）是一种用于缓解测试预言问题（Test Oracle Problem）的测试方法^[37-39]。它并不要求测试者事先为每个输入给出精确的预期输出，而是通过构造相关输入之间的语义联系，并检查相应输出是否满足预期约束，从而间接判断被测程序是否可能存在缺陷。

2.3.1 测试预言问题与蜕变测试的基本思想

在传统软件测试中，测试者通常需要针对每个输入事先给出预期输出，并通过比较程序实际输出与预期输出是否一致来判断程序行为是否正确。然而，在许多实际场景中，程序的正确输出难以直接获得，或者即使理论上存在正确结果，也难以以低成本方式构造和校验。这种难以为给定输入确定可靠预期输出的情形，通常被称为测试预言问题^[23]。

对于程序分析工具而言，测试预言问题往往更加突出。以程序切片工具为例，其输出通常表现为一组语句、依赖关系或结构化程序片段，而不是单一数

值。由于分析结果本身具有较强语义性，且不同工具、不同实现层级下的输出形式可能存在差异，研究者往往难以针对大量测试样例逐一构造精确且可自动校验的标准答案。在这种情况下，直接依赖“输入—预期输出”模式开展测试将面临较大困难。

蜕变测试的基本思想并不是为单个输入显式构造精确输出，而是利用被测对象应满足的某种性质，构造多个相关测试用例，并通过检查这些测试结果之间是否满足预期关系来间接判断程序行为是否合理^[37,39]。因此，蜕变测试关注的并非某一次执行结果本身是否等于预期值，而是相关执行结果之间是否满足应有的语义约束。

在蜕变测试中，通常将初始输入称为源测试用例（source test case），由源测试用例经过某种变换得到的输入称为后续测试用例（follow-up test case）。相应地，源测试用例和后续测试用例所对应的输出，则共同构成蜕变关系的检验对象。只要被测程序在这些相关输入上的输出违反了预先设定的关系约束，就可以将其视为异常信号。

2.3.2 蜕变关系与任务实例化

蜕变关系（Metamorphic Relation, MR）是蜕变测试的核心组成部分，用于描述源测试用例与后续测试用例之间，以及相应输出之间应满足的约束关系^[37,39]。蜕变测试能否有效发现缺陷，在很大程度上取决于蜕变关系是否合理、清晰且可执行。

一般而言，蜕变关系可以看作由两个部分组成：一部分规定如何从源测试用例构造后续测试用例，另一部分规定这些相关测试结果之间应满足何种关系。若被测程序在测试过程中违反该关系，则说明其行为与预期性质不一致。

从形式上看，设被测程序为 f ，源测试输入为 x ，由某种输入变换 t 得到的后续测试输入为 $x' = t(x)$ ，则相应输出分别为 $f(x)$ 和 $f(x')$ 。若某一性质要求这两个输出之间满足关系 R ，则可以将蜕变关系的一般形式写为：

$$x' = t(x) \Rightarrow R(f(x), f(x')). \quad (2-1)$$

上述表达式表明，蜕变关系并不要求事先知道 $f(x)$ 或 $f(x')$ 的精确值，而

是要求在输入满足特定变换关系时，输出之间也应满足相应约束。正因为如此，蜕变关系为缺少直接测试预言的测试对象提供了一种可操作的间接判定依据。

蜕变关系并非固定不变的通用模板，而是与具体问题领域密切相关的测试约束。不同被测对象往往对应不同类型的蜕变关系。例如，有些关系体现为输入经过语义保持变换后输出应保持不变，有些关系体现为在加入无关成分后输出的核心性质仍应稳定，还有些关系则体现为多个测试结果之间应满足一致性、包含性或单调性等约束^[39]。

任务实例化是指将抽象蜕变关系进一步落实为具体程序变换、执行与结果比较任务的过程。围绕不同关系类型，可以形成不同的输入构造方式、适用前提和结果检查机制。

2.3.3 蜕变测试流程

蜕变测试通常围绕源测试用例、后续测试用例以及二者之间的关系检验展开，其基本流程可概括为以下几个步骤：首先，选择一个源测试用例并执行被测程序，获得源测试结果；其次，根据预先定义的输入变换规则构造一个或多个后续测试用例；再次，执行后续测试用例并获得相应结果；最后，根据蜕变关系检查这些执行结果之间的约束是否成立。若关系成立，则说明当前测试未发现异常；若关系被违反，则说明被测程序可能未满足该问题应具有的性质，从而可能存在缺陷^[37,39]。

2.4 测试用例生成技术

测试用例生成是软件测试中的重要环节，其目标是构造能够有效触发程序行为并支持后续分析与验证的输入样例。对于程序分析工具测试而言，测试对象本身往往是待分析程序、程序片段或具有特定语义结构的代码样本。因此，测试用例生成在这里主要体现为程序样例的生成。

与本研究相关的程序样例生成方式主要包括两类：一类是基于 Csmith 的随机程序生成，另一类是基于大语言模型提示约束的定向生成。前者强调自动化规模、结构多样性与语法可靠性，后者强调围绕特定语义结构进行目标化构造。

2.4.1 基于 Csmith 的随机程序生成

随机程序生成是一类依据语言语法、类型约束和生成策略自动构造程序样例的技术。与人工逐条编写测试程序不同，这类方法能够在较短时间内生成大量结构各异的程序，从而为软件测试提供规模化输入来源^[25-26]。在程序分析与编译器测试研究中，随机程序生成不仅用于扩大输入空间，也常被用来系统覆盖不同的语法结构、表达式组合和控制流模式。

Csmith 是面向 C 语言的代表性随机程序生成工具，其核心目标是自动生成语法正确、可独立编译执行并尽量规避未定义行为的 C 程序^[25]。为此，Csmith 在生成过程中会同时维护语法合法性、类型一致性以及若干与 C 语言语义相关的约束条件，例如避免生成明显未初始化使用、越界访问或未受控副作用等易导致未定义行为的结构。生成结果通常是自包含的完整程序，包含全局定义、局部变量、表达式、控制结构以及必要的返回与输出逻辑，因此便于直接进入后续编译与执行流程。

从程序结构上看，Csmith 并非简单拼接语句模板，而是围绕表达式、语句、控制流和数据对象逐层展开生成。其输出程序通常包含算术表达式、条件分支、循环、数组、结构体和指针等常见 C 语言构造，并通过内部约束机制控制这些构造之间的组合方式。正因为生成过程同时考虑了语法正确性与基本语义合法性，Csmith 能够在保持自动化规模的同时，为测试提供较为稳定的程序样例来源。

与 Csmith 类似，YARPGen 也是面向 C/C++ 程序测试的重要随机生成工具^[26]。不同之处在于，Csmith 更强调通用 C 程序结构的随机覆盖，而 YARPGen 更侧重围绕特定优化敏感模式生成测试程序。

2.4.2 基于大语言模型提示约束的定向生成

与随机生成方式不同，基于大语言模型的程序生成更强调通过提示约束直接控制样例的语义结构。这类方法能够围绕多级指针、复杂别名、跨函数数据传播以及局部变量的形成与传播路径，有针对性地构造程序样例。

下面给出一个面向相关程序结构的提示词示例：

- 可独立编译运行；
- 包含数组访问、指针别名或二级指针；

- 存在跨函数数据传播;
- 在返回语句前保留一个经过前序数据传播形成的局部变量, 例如 `keep`;
- 不依赖外部输入, 不包含未定义行为;
- 代码简短, 但依赖链不能过浅。

在上述提示约束下, 大语言模型可以生成如例 2.1 所示的程序。

```
1  #include <stdio.h>
2
3  static void propagate(int **pp, int delta) {
4      **pp = **pp + delta;
5  }
6
7  static int build_keep(int *arr, int idx) {
8      int x = arr[idx];
9      int *p = &x;
10     int **pp = &p;
11
12     propagate(pp, 2);
13
14     int keep = x;
15     return keep;
16 }
17
18 int main(void) {
19     int data[3] = {3, 5, 7};
20     int result = build_keep(data, 1);
21     printf("%d\n", result);
22     return 0;
23 }
```

例 2.1 基于提示约束的大语言模型生成程序示例

该示例包含数组读取、局部别名建立、二级指针跨函数写回以及返回值传播等结构。程序中的局部变量取值不是由单条赋值语句直接给出, 而是经过“数组访问 → 局部别名建立 → 二级指针跨函数写回 → 返回值传播”的路径形成, 从而体现了提示约束下程序样例的定向构造特征。

2.4.3 两类生成方式的特征差异

随机生成与提示约束生成在程序样例构造方式上存在明显差异。前者主要依赖生成器内部的语法与语义约束自动扩展输入空间，更适合持续提供数量可观、结构多样的程序样例；后者则依赖外部提示直接约束程序结构，更适合围绕给定语义目标构造具有特定依赖特征的程序。二者分别对应规模化随机覆盖与目标化定向构造两类不同的程序生成思路。

2.5 本章小结

本章对后续研究所涉及的相关概念与技术基础进行了系统梳理。首先，介绍了程序切片的定义、分类、切片准则、数据依赖与控制依赖，并进一步说明了指针别名分析对数据依赖提取与切片精度的影响；随后，介绍了程序依赖图与系统依赖图的基本作用，以及源码级与中间表示级切片工具的实现路径差异，重点说明了 `Frama-C` 与 `dg` 分别对应的源码级与 `IR` 级实现特征；接着，介绍了蜕变测试的基本思想、蜕变关系及其任务实例化过程，并给出了蜕变测试的一般流程；最后，讨论了基于 `Csmith` 的随机程序生成与基于大语言模型提示约束的定向生成两类测试输入构造技术。上述内容为第三章的蜕变关系设计、第四章的方法框架以及第五章的实验分析提供了概念与技术基础。

第三章 面向 C 程序切片工具的蜕变关系设计

本章围绕 C 程序切片工具测试中的预言缺失问题，讨论面向切片语义的领域专用蜕变关系设计。围绕程序切片中具有代表性的几类风险，本章从设计目标与原则出发，分别讨论四类蜕变关系的关系定义、理论依据、可执行判定原则、构造方式、示例及其适用范围。

为便于后续关系定义，本文将切片工具输出经位置映射、层级统一和无关噪声过滤后得到的可比较结果称为“整理后切片结果集合”，并记为 $\text{NormSlice}(P, C)$ 。其中， P 表示待切片程序， C 表示切片准则。本章仅从关系定义与判定条件角度使用该抽象概念，具体的结果整理流程将在第四章中说明。

3.1 蜕变关系设计目标与原则

面向 C 程序切片工具设计蜕变关系时，需要围绕切片结果的语义约束构造可执行、可比较的验证关系。关系设计应直接对应切片工具在准则相关语义保持、无关成分排除以及多准则处理一致性等方面的基本要求。

面向 C 程序切片工具的验证任务，蜕变关系设计至少应满足以下五项原则。

1. **准则相关性。**关系必须直接对应某种切片语义期望，即其比较对象应围绕同一切片准则展开，能够清楚回答哪些语句本应被保留、哪些语句本应被剔除。
2. **贴合 C 语言分析特征。**关系设计应围绕 C 程序中具有代表性且会显著影响分析结果的语义结构展开，如指针与数组访问、结构体字段访问、局部数据流链以及显式控制分支等，以刻画切片工具在数据依赖与控制依赖分析中的潜在薄弱环节。
3. **测试任务构造合法且可比较。**对需要派生程序的蜕变关系，所构造的派生程序应满足明确的语义前提，符合 C 语言语法与基本语义约束，并能够被后续切片与比较流程稳定处理；对不需要派生程序的蜕变关系，则应能够

基于同一源程序稳定构造出一组可比较的切片任务。

4. **关系违背具有明确指向性。**蜕变关系的成立或违背应尽量对应清晰的语义含义，使判定结果能够进一步指向语义保持失败、无关扰动残留或多准则组合不一致等不同问题，而不是仅得到缺乏分析指向性的二元结论。
5. **抽象定义与可执行判定一致。**蜕变关系既要在理论上对应清晰的切片语义，也要在工程上能够落实为可执行判定原则。这意味着蜕变关系既要能够描述切片语义约束，也要能够在后续自动化测试框架中转化为可执行的结果整理、关系检查与状态输出步骤。

表 3-1 四类蜕变关系的设计目标、可执行判定原则与主要风险

关系	设计目标	可执行判定原则	主要针对的风险
MR1	检验准则相关语义保持条件下切片结果是否一致	比较源程序与派生程序在统一观测点上的切片后行为是否保持一致	工具对语义等价变换处理不稳定，导致切片结果失去语义保持性
MR2	检验与切片准则无关的数据流成分是否被错误保留	检查新增数据流对应的位置集合是否仍与整理后的切片结果相交	数据依赖分析不精确，导致无关数据流残留于切片结果中
MR3	检验与切片准则无关的控制流成分是否被错误保留	检查不可达控制负载对应的位置集合是否仍与整理后的切片结果相交	控制依赖或可达性处理不准确，导致无关控制结构未被剔除
MR4	检验多准则切片结果是否满足预期的组合一致性关系	比较单变量切片结果集合与多变量切片结果集合之间的并集等式或包含关系	多准则结果处理不一致，导致组合切片结果出现异常扩张或遗漏

表 3-1 给出了四类蜕变关系的设计目标、可执行判定原则及其主要风险。总体而言，MR1 关注准则相关语义保持，MR2 和 MR3 分别关注数据非干扰与控制非干扰，MR4 关注多准则组合一致性。四类关系共同覆盖切片工具中依赖恢复、无关成分排除和组合任务处理等典型风险。

3.2 MR1：面向语义保持的蜕变关系设计

3.2.1 设计动机

MR1 面向语义保持场景，关注程序在切片准则相关语义不变时，切片结果是否仍保持应有的一致性。该关系对应的风险在于：当相关路径发生等价改写

后，工具可能在依赖分析、准则映射或结果导出过程中表现出不稳定性，从而使切片结果偏离原有的准则相关行为。

MR1 的构造重点是围绕切片准则相关计算路径实施语义保持改写，并考察工具在相关路径结构发生变化后，是否仍能稳定保留准则相关行为。与后续关系相比，MR1 关注相关路径重写后的结果一致性；MR2 与 MR3 分别关注无关数据流和无关控制流是否会被错误保留。

3.2.2 关系定义、理论依据与可执行判定原则

设源程序为 P ，通过语义保持变换得到的派生程序为 P' ，切片准则分别为 C 和映射到派生程序上的 C' 。若 P 与 P' 在切片准则相关语义上等价，则应满足如下关系：

$$P \equiv_C P' \Rightarrow \text{Slice}(P, C) \equiv_C \text{Slice}(P', C'). \quad (3-1)$$

其中， $P \equiv_C P'$ 表示两个程序在切片准则所关注的可观察行为上保持一致； $\text{Slice}(\cdot)$ 表示围绕给定切片准则执行切片。式 (3-1) 给出了 MR1 的抽象关系定义。该关系并不要求两个切片结果在文本形式上完全相同，而要求它们在切片准则相关语义上保持一致。

该关系的理论依据在于：后向切片的目标是保留所有可能影响切片准则处观测行为的依赖成分。若程序变换没有改变准则相关变量的取值来源、控制条件和过程间传播关系，则准则相关依赖结构在语义层面应保持一致。因此，切片工具在源程序与派生程序上得到的结果虽然不必在文本形式上完全相同，但不应导致切片后程序在统一观测点上的行为差异。

在执行层面，统一观测点上的行为一致性作为判定依据。只有当源程序与派生程序在未切片版本上表现一致时，才将其视为有效的 MR1 实例；在此基础上，若二者在切片后仍保持一致，则认为关系成立，否则记为关系违背。因此，MR1 的可执行判定原则建立在“未切片一致、切片后仍应一致”的比较方式之上。

3.2.3 派生程序设计

MR1 的设计重点在于围绕切片准则相关计算路径实施语义保持型重写。此类派生程序应同时满足两个条件：其一，不改变切片准则相关变量在观测点处的

取值；其二，在语法结构、局部依赖链或内存访问形式上形成足够明显的差异。

在构造层面，可将相关改写分为基础改写与扩展改写两类。基础改写用于建立较易验证的语义保持关系，主要包括：

1. **数组访问与指针访问的等价改写**。例如将 `a[i]` 改写为 `*(a+i)`，或显式引入中间指针变量。这类变换直接涉及 C 语言中的地址计算与解引用。
2. **结构体字段访问的等价改写**。例如将箭头字段访问改写为等价的显式解引用字段访问，用于检验工具对结构体指针访问的依赖建模是否稳定。
3. **局部表达式分解与临时变量传递**。例如将 `x = a[i] + 1` 分解为“先读取、再经临时变量传递、最后赋值”，在不改变结果的前提下拉长局部 def-use 链。

在基础改写之上，还可采用基于真实函数调用嵌入的语义保持改写。其基本思路是：先定位切片准则相关路径上的活跃表达式位置，再利用运行信息收集该位置的取值与局部环境，随后将真实函数调用嵌入原有计算路径，最后通过补偿项保持观测点处的值与原程序一致。这样的改写不以简单插入额外代码为目标，而是通过更显著的结构性重写检验工具在相关路径变化下的稳定性。

3.2.4 C 语言示例

下面给出一组 MR1 的 C 语言等价程序示例，如例 3.1(a) 与例 3.1(b) 所示。

```

1  int load_keep(int *a, int i) {
2      int keep = a[i] + 1;
3      return keep;
4  }
```

例 3.1(a) MR1 的原程序 *P*

该示例围绕变量 `keep` 的计算路径构造语义保持关系：原程序采用直接表达式计算，而派生程序在同一路径上引入真实函数调用并通过补偿保持观测点处取值一致。若切片准则取为“在 `return keep;` 处观测变量 `keep`”，则两个程序在切片准则相关语义上应保持一致。派生程序直接重写了 `keep` 的相关计算路径，引入了函数调用边界，并通过补偿项使观测点处的最终值恢复为原表达式结果。因此，这类“相关路径已重写但准则相关可观察行为保持不变”的程序对，可用于检验切片工具在语义保持场景下的稳定性。


```

1  static int rw_digits(char *buf, int max, unsigned int value) {
2      int k = 0;
3      while (value && k < max) {
4          buf[k++] = '0' + value % 10;
5          value /= 10;
6      }
7      return k;
8  }
9
10 int load_keep(int *a, int i) {
11     char h[4] = {0, 0, 0, 0};
12     int keep = a[i] + 1 + (rw_digits(h, i + 4 - i, 48u + i - i) - 2);
13     return keep;
14 }

```

例 3.1(b) MR1 的派生程序 P'

3.2.5 适用范围与局限

MR1 适合检验切片工具在语义保持场景下的稳定性，尤其适合暴露工具对局部依赖重构、函数调用边界变化以及内存访问形式变化的敏感性。然而，MR1 的有效性建立在“源程序与派生程序在切片准则相关语义上确实等价”这一前提之上。若样例构造本身破坏了语义保持前提，则后续比较即使出现差异，也不能直接归因于切片工具错误。

MR1 的适用边界更接近可验证的观测等价，而不是对完整程序语义等价的形式化证明。与 MR2/MR3 所针对的无关扰动残留问题不同，MR1 的核心判据始终是：相关路径可以被显著重写，但切片准则相关可观察行为应保持不变。

3.3 MR2 与 MR3 的共享构造流程

尽管 MR2 与 MR3 的关系前提不同，但二者在构造层面共享相近的派生程序生成思路：均围绕既定切片准则选择插入位置，在程序中引入一段可定位的新增成分，并在切片完成后检查该新增成分是否仍被错误保留。抽象地看，这一流程可归纳为插入位置选择、新符号构造以及关系特定实例化三个步骤。表 3-2 总结

表 3-2 MR2 与 MR3 的主要构造方式及示例模板

关系	构造方式	示例模板	判定要求
MR2	插入无关局部数据流链	<pre>int mt_a = i + 1; int mt_b = mt_a * 2;</pre>	新增数据流对应的位置集合不应与整理后的切片结果相交
	插入无关数组/指针链	<pre>int mt_buf[2]; int *mt_p = &mt_buf[1];</pre>	
	插入无关辅助函数调用	<pre>int mt_sink = mt_helper(i);</pre>	
MR3	恒假条件分支	<pre>if (0) { stmt; } if (false) { stmt; }</pre>	不可达控制负载对应的位置集合不应与整理后的切片结果相交
	矛盾条件分支	<pre>if (a < 0 && a > 100) { stmt; }</pre>	
	零迭代循环	<pre>while (0) { stmt; }</pre>	

了二者的主要构造方式、示例模板及判定要求。

算法的核心步骤可以概括为：先确定插入位置，再构造新符号集合，最后依据关系类型生成具体新增成分。对 MR2，新增成分应满足与切片准则无反向数据依赖关系；对 MR3，新增成分应位于控制上不可达的路径中。算法 3-1 给出了这一共享流程。

算法 3-1 MR2 与 MR3 的共享构造流程

输入： 程序 P ，切片准则 C ，关系类型 $r \in \{\text{MR2}, \text{MR3}\}$

- 1: 选取与切片准则 C 无关、即不直接改变准则相关计算路径的插入位置 s ；
- 2: 创建与切片准则无别名关系的新符号集合 U ，并记录其在当前判定层级下对应的位置集合 L_U ；
- 3: **if** $r = \text{MR2}$ **then** 基于 U 构造不向切片准则传播数据依赖的新增成分 q ；
- 4: **else** 基于 U 及局部上下文构造位于控制上不可达路径中的新增成分 q ；
- 5: 将新增成分 q 插入位置 s ，生成派生程序 P' ；
- 6: 记录关系类型、插入位置、新增符号集合及其位置集合，形成后续判定与结果回溯所需的构造信息 M 。

输出： 派生程序 P' 、新增符号集合 U 、新增位置集合 L_U 以及构造信息 M

在关系判定阶段，若新增成分对应的位置集合仍与整理后的切片结果相交，则视为关系违背；若二者不相交，则视为关系满足；若切片任务本身无法成功完成，则单独记录为执行失败。二者共享相同的判定框架，但分别对应数据依赖无

关性与控制可达性无关性两类不同前提。

3.4 MR2：面向无关数据流扰动的蜕变关系设计

3.4.1 设计动机

MR2 关注的问题是：当程序中引入一段与切片准则无关、但位于可执行路径上的新增数据流时，切片工具能否将其正确排除在切片结果之外。该关系对应的风险在于，工具可能将本不应纳入准则相关计算的局部数据流错误地视为相关，从而暴露出数据依赖分析不精确、别名处理过于保守或切片结果裁剪不充分等问题。

3.4.2 关系定义、理论依据与可执行判定原则

设 P' 为在源程序 P 中插入一组新增语句后得到的派生程序， U 表示这些新增语句中引入的新符号集合， L_U 表示这些新增语句在当前判定层级下对应的位置集合。若该新增语句片段不向切片准则 C 传播数据依赖，则其对应位置不应再出现在整理后的切片结果中，即应满足

$$\text{Dep}_d(U, C) = \emptyset \Rightarrow L_U \cap \text{NormSlice}(P', C) = \emptyset. \quad (3-2)$$

其中， $\text{Dep}_d(U, C) = \emptyset$ 表示新增符号集合 U 不存在通向切片准则 C 的反向数据依赖路径； $\text{NormSlice}(P', C)$ 表示对切片工具原始输出进行位置映射、层级统一和必要过滤后得到的整理后切片结果位置集合。对源码级工具，该集合可具体表现为源码语句位置集合；对 IR 级工具，则可具体表现为 IR 指令或基本块位置集合。式 (3-2) 表明，若新增语句只形成一段与切片准则无关的局部数据流，则这些新增语句对应的位置不应出现在最终切片结果中。

该关系的理论依据在于：后向切片从切片准则出发，沿数据依赖和必要的控制依赖反向追溯相关语句。若新增数据流仅在新鲜局部对象内部传播，且不存在通向切片准则的反向数据依赖路径，则该新增片段不会影响准则处变量的取值形成过程。因此，在理想切片语义下，其对应位置不应出现在准则相关切片结果中。

MR2 的可执行判定原则可归结为“新增数据流对应位置集合是否仍与整理后的切片结果相交”。若二者仍相交，则说明工具错误地保留了与准则无关的数据流成分，从而构成 MR2 违例。

3.4.3 派生程序设计

在共享构造流程的基础上，MR2 的派生程序设计重点是在可执行路径上插入一段与切片准则无反向数据依赖的新增数据流。新增符号应保持新鲜，不与准则变量或其存储位置形成别名关系；新增语句的值传播应局限于新引入的局部对象内部，不进入准则相关路径。

本文主要通过局部变量链、数组/指针链和局部辅助函数调用构造此类新增数据流。判定时，以新增语句对应的位置集合是否仍出现在整理后的切片结果中作为观察依据。

3.4.4 C 语言示例

```
1  int load_keep(int *a, int i) {
2      int keep = a[i] + 1;
3      return keep;
4  }
```

例 3.2(a) MR2 的原程序 P

```
1  int load_keep(int *a, int i) {
2      int keep = a[i] + 1;
3
4      int mt_buf[2];
5      mt_buf[0] = i;
6      mt_buf[1] = i + 1;
7      int *mt_p = &mt_buf[1];
8      int mt_sink = (*mt_p) * 2 + 1;
9      (void)mt_sink;
10
11     return keep;
12 }
```

例 3.2(b) MR2 的派生程序 P'

如例 3.2(a) 与例 3.2(b) 所示。该示例在不改变切片准则变量 `keep` 的前提下，于可执行路径上引入一段围绕局部对象的新增数据流；该新增数据流会被实际执行，但不会改变 `return keep;` 处 `keep` 的取值。

若切片准则取为“在 `return keep;` 处观测变量 `keep`”，则 `mt_buf`、`mt_p` 与 `mt_sink` 对应的新增位置均不应在最终整理后的切片结果中出现。它们位于真实可执行路径上，但其值传播仅发生在新引入的局部对象内部，不会形成通向切片准则的反向数据依赖，也不会改变 `return keep;` 处 `keep` 的取值。若工具仍将该局部数组/指针链整体保留，则通常说明其在新增内存访问的依赖传播上过于保守，或者未能在结果导出阶段将与准则无关的成分充分剔除。

3.4.5 适用范围与局限

MR2 主要适用于检验切片工具对**无关但可执行的数据流成分**的排除能力。其关注点在于数据依赖是否被错误扩张，而不在于控制路径本身是否可达。因此，若新增成分之所以不应保留，主要原因在于其位于控制上不可达的路径中，而非因其不向切片准则传播数据依赖，则更适合交由 MR3 处理。

MR2 与 MR3 虽然都可通过“新增成分是否仍被保留”来进行观察，但二者所对应的理论前提并不相同：前者针对数据依赖无关性，后者针对控制可达性无关性。

3.5 MR3: 面向无关控制流扰动的蜕变关系设计

3.5.1 设计动机

MR3 关注的问题是：当程序中引入一段控制上不可达的新增结构时，切片工具能否将其正确排除在切片结果之外。该关系对应的风险在于，不可达路径中的语句虽然在运行时不会被执行，但在表面形式上仍可能与准则相关路径相近，例如包含指针访问、结构体字段读取或对准则相关变量的引用。若工具仅依据局部语法邻近、变量共现或粗粒度控制依赖近似进行保留，就可能将这些本不应出现于切片结果中的语句错误带入结果。

3.5.2 关系定义、理论依据与可执行判定原则

设源程序为 P ，在其中插入一段控制上不可达的包装结构后得到派生程序 P' 。记该不可达包装及其内部负载所引入的新鲜符号集合为 U ，其在当前判定层级下对应的位置集合为 L_U ，切片准则为 C 。若该新增结构在控制上对切片准则不可达，则应满足

$$\text{Reach}_c(U, C) = \emptyset \Rightarrow L_U \cap \text{NormSlice}(P', C) = \emptyset. \quad (3-3)$$

其中， $\text{Reach}_c(U, C) = \emptyset$ 表示从控制可达性角度看，新增结构不可能参与影响切片准则相关计算； $\text{NormSlice}(P', C)$ 表示经位置映射、层级统一和必要过滤后得到的整理后切片结果位置集合。式 (3-3) 表明，若新增控制结构在语义上不可达，则其内部负载对应的位置不应残留于最终切片结果中。

该关系的理论依据在于：若新增负载位于语义上不可达的控制路径中，则该路径中的语句不会在任何实际执行中改变切片准则处的变量取值或执行行为。对于需要排除不可达成分的切片结果而言，这类负载不应被纳入准则相关切片。若工具仍将其保留，则通常反映出控制可达性处理、控制依赖裁剪或结果导出阶段存在过度保留问题。

MR3 的可执行判定原则与 MR2 在形式上相似，但理论依据不同：MR2 的前提是数据依赖无关，MR3 的前提是控制可达性无关。若不可达路径中的新增语句仍出现在整理后的切片结果中，则说明工具在控制依赖传播、可达性处理或结果导出方面可能存在偏差。

3.5.3 派生程序设计

在共享构造流程的基础上，MR3 的派生程序设计重点在于构造一段控制上不可达但表面形式上具有迷惑性的新增负载。为了增强对工具控制依赖处理能力的施压，新增负载可以包含局部依赖链、指针访问、结构体字段读取或与准则变量相邻的局部计算，但其外层包装必须保证该路径在语义上不可达。

MR3 主要使用恒假条件分支、矛盾条件分支、零迭代循环以及非命中 switch 分支等模板构造不可达路径。这些构造的共同特点在于：控制包装在理论上不应被执行，但其内部语句在语法表面上足以与真实路径形成相似痕迹，从而更容易

暴露工具是否存在“仅依据表面结构近似保留”的问题。判定时，同样以新增负载对应的位置是否仍出现在整理后的切片结果中作为直接观察依据。

3.5.4 实例化模式与 C 语言示例

下面给出一组适用于 MR3 的 C 语言示例。例 3.3(a) 为原程序，例 3.3(b) 为在原程序中插入不可达控制负载后的派生程序。该示例的关键在于：新增代码片段在表面上与准则相关路径相邻，并包含指针访问与局部计算，但其外层条件恒为假，因此在语义上不应参与 keep 的计算。

```
1 int load_keep(int *a, int i) {
2     int keep = a[i] + 1;
3     return keep;
4 }
```

例 3.3(a) MR3 的原程序 P

```
1 int load_keep(int *a, int i) {
2     int keep = a[i] + 1;
3
4     if (0) {
5         int mt_tmp[2];
6         mt_tmp[0] = i;
7         mt_tmp[1] = i + 3;
8         int *mt_q = &mt_tmp[1];
9         keep = (*mt_q) + 1;
10    }
11
12    return keep;
13 }
```

例 3.3(b) MR3 的派生程序 P'

若切片准则取为“在 `return keep;` 处观测变量 `keep`”，则上述恒假分支中的新增语句对应的位置均不应在最终整理后的切片结果中出现。尽管该分支内部显式引用了 `keep`，并包含指针访问和局部依赖链，但由于其控制条件恒假，该路径在语义上不可达，不应影响准则相关行为。若工具仍将该负载带入切片结果，则通常说明其在控制可达性处理或控制依赖裁剪方面存在偏差。

3.5.5 适用范围与局限

MR3 主要适用于检验切片工具对**控制上不可达负载**的排除能力。其核心在于：新增结构不应因为表面形式上与真实路径相近，或因为局部变量与准则变量发生共现，就被错误纳入切片结果。因此，MR3 尤其适合暴露控制依赖传播过宽、死路径识别不充分以及结果导出不彻底等问题。

不过，MR3 的有效性建立在“不可达路径的构造前提明确成立”这一条件之上。若某些控制包装在不同编译选项、宏展开条件或工具内部近似模型下不再保持稳定的不可达性，则关系判定的解释力度会受到影响。因此，MR3 更适合采用语义清晰、人工可验证且不依赖复杂外部条件的不可达模板。

需要说明的是，部分静态切片工具可能采用较保守的路径近似策略，并不承诺完全剔除所有不可达代码。因此，MR3 的违例在解释时应结合工具声明的分析策略进行复核；在本文实验中，该关系主要用于揭示控制可达性处理和结果裁剪方面的潜在精度问题，而不直接等同于对所有静态切片语义的健全性否定。

3.6 MR4：面向切片准则组合一致性的蜕变关系设计

3.6.1 设计动机

与前 3 类关系不同，MR4 并不围绕源程序与派生程序之间的关系展开，而是关注同一程序在多个切片准则下的结果是否保持应有的一致性。若分别围绕变量 x 与变量 y 构造切片，再将二者结果合并，则该合并结果应与围绕准则组合 $\{x, y\}$ 直接执行切片得到的结果一致。若这一关系不成立，则说明工具在多准则切片处理、结果合并或导出阶段可能存在不自洽问题。

MR4 关注的是切片工具在准则组合层面是否满足基本一致性。前 3 类关系围绕程序变换和新增扰动展开，MR4 则围绕同一程序上的多任务组织展开。

3.6.2 关系定义、理论依据与可执行判定原则

设程序为 P ，程序点为 ℓ ，变量 x 与 y 均在该程序点可观测。记围绕单变量准则 $(\ell, \{x\})$ 与 $(\ell, \{y\})$ 得到的整理后切片结果分别为 S_x 与 S_y ，围绕组合准则

$(\ell, \{x, y\})$ 得到的整理后切片结果为 $S_{x,y}$ 。则 MR4 要求满足如下组合一致性关系：

$$S_{x,y} = S_x \cup S_y. \quad (3-4)$$

式 (3-4) 描述的是多准则切片组合一致性的基本要求：当两个变量在同一程序点上被联合纳入切片准则时，组合切片结果应等于相应单变量切片结果的并集。在工程上，这一关系直接落实为对整理后结果集合的并集等式检查。若 $S_{x,y}$ 与 $S_x \cup S_y$ 不相等，则说明工具在多准则任务的内部处理与结果导出之间可能存在不一致。

该关系的理论依据在于：多变量后向切片可以看作从多个准则变量同时出发，对其相关依赖进行闭包计算。若工具对单变量准则和组合准则采用一致的依赖追踪与结果整理策略，则组合准则得到的切片结果应等价于各单变量切片结果的并集。因此，式 (3-4) 可用于检验多准则处理、结果合并和导出过程是否自洽。

MR4 不是通过检验程序运行后的具体观测值，而是检验切片结果集合的组织方式是否自洽，因此更适合作为任务级一致性关系，其与运行语义关系有本质区别。

3.6.3 切片任务设计

MR4 不需要显式构造派生程序，其核心在于围绕同一源程序构造一组彼此可比较的切片任务。具体而言，围绕同一程序点上的两个可观测变量 x 与 y ，构造三组切片任务：围绕 $(\ell, \{x\})$ 的单变量任务、围绕 $(\ell, \{y\})$ 的单变量任务，以及围绕 $(\ell, \{x, y\})$ 的组合任务。三者共享同一程序与同一观测点，从而减少程序结构差异带来的额外干扰。

MR4 通常围绕同一程序点上共同出现且均具有明确数据来源的两个变量构造单变量任务与组合任务。例如，在返回语句、输出语句或显式组合表达式所在位置选择两个变量作为切片准则，可以保证组合任务与单变量任务之间具有明确可比性。为减少额外干扰，所选变量应能够在各自比较层级上形成稳定结果集合，并尽量避免宏展开、未映射语句或工具特定导出噪声对集合比较造成影响。

3.6.4 C 语言示例

下面给出一组适用于 MR4 的 C 语言示例。假设源程序如例 3.4 所示：

```
1  int main(void) {  
2      int a = 3, b = 4, c = 5;  
3  
4      int base = a + b;  
5      int noise = c;  
6      int noise2 = noise + 1;  
7      int x = base * 2;  
8      int y = base - 1;  
9      printf("%d %d\n", x, y);  
10     return 0;  
11 }
```

例 3.4 MR4 的示例程序 *P*

可以围绕该程序构造三个切片任务：相对于变量 *x* 的单变量切片、相对于变量 *y* 的单变量切片，以及相对于变量集合 {*x*, *y*} 的组合切片。

在该示例中，变量 *x* 与 *y* 共享语句 `base = a + b;`，因此二者的单变量切片结果在公共依赖处会发生重合。理论上，相对于 *x* 的切片应保留 *base*、*x* 以及观测语句；相对于 *y* 的切片应保留 *base*、*y* 以及观测语句；相对于 {*x*, *y*} 的组合切片，则应同时保留 *base*、*x*、*y* 以及观测语句。也就是说，组合切片结果应等于两个单变量切片结果的并集。

与此同时，变量 *noise* 及其局部无关计算不参与 *x* 和 *y* 的取值形成，因此不应出现在上述切片结果中。若组合切片结果与两个单变量切片结果的并集不一致，或错误保留了无关语句，则说明工具在多准则处理或结果导出过程中可能存在不一致问题。

3.6.5 适用范围与局限

MR4 适合检验支持多变量准则的切片工具是否满足组合一致性，尤其适合揭示多变量依赖传播、结果合并以及切片导出过程中的结果组织不一致问题。然而，MR4 的有效性建立在“结果已经按照各自层级整理为可比较的位置集合”这一前提之上。若不同工具的输出层级不同，或结果整理过程本身存在歧义，则集

合比较的解释力度会受到影响。

MR4 关注的是切片结果集合的组织方式是否自治，而非程序执行后的具体观测值是否一致。正因为如此，MR4 与前 3 类关系形成互补：前者强调组合一致性，后者强调程序变换下的语义保持与无关扰动隔离。

3.7 本章小结

本章围绕面向 C 程序切片工具的自动化验证需求，设计了四类领域专用蜕变关系。其中，MR1 用于检验语义保持变换下的切片稳定性，MR2 与 MR3 分别用于检验无关数据流成分和无关控制流成分是否会被错误保留，MR4 用于检验多准则切片结果是否满足组合一致性。

围绕这四类关系，本章分别说明了其设计动机、关系定义、理论依据、可执行判定原则、构造方式、C 语言示例以及适用范围。总体来看，四类关系分别从依赖保持、数据非干扰、控制非干扰和准则组合一致性角度刻画切片工具的潜在风险，将通用蜕变测试思想转化为面向 C 程序切片工具的可执行判定条件。这为后续自动化测试框架构建、实验设计、结果统计与案例分析提供了关系基础。

第四章 面向 C 程序切片工具的蜕变测试框架

第三章设计了面向 C 程序切片工具的领域专用蜕变关系，为后续自动化判定提供了关系基础。本章在此基础上构建蜕变测试框架，说明如何组织种子程序准备、任务实例化、切片执行、结果整理与关系判定等过程，从而支持源码级和 LLVM IR 级切片工具的统一验证。

4.1 框架概述

4.1.1 整体框架简介

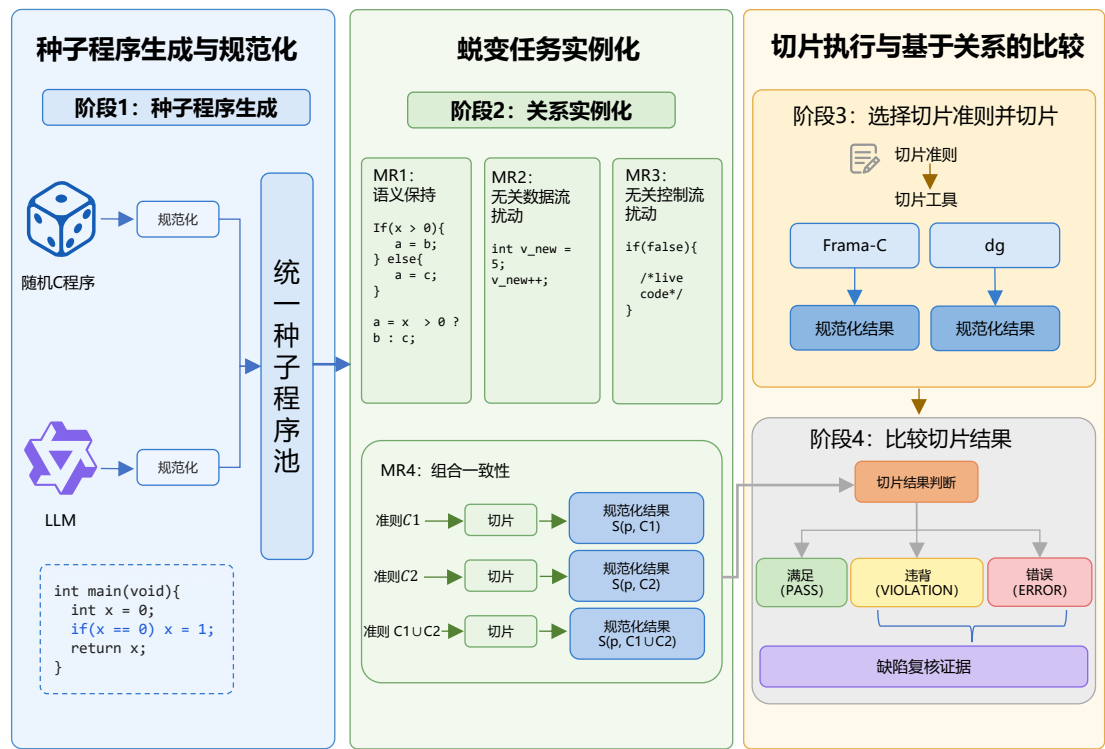


图 4-1 面向 C 程序切片工具的蜕变测试框架

图 4-1 给出了本文采用的蜕变测试框架。该流程从左到右划分为输入准备、任务构造和执行与比较三个功能区域，并进一步组织为种子程序准备、蜕变任务

实例化、切片准则选择与切片执行、分层结果整理与关系判定四个阶段。第一阶段将原始程序整理为统一的种子程序对象；第二阶段围绕第三章提出的蜕变关系构造测试任务；第三阶段确定切片准则并执行切片；第四阶段按照工具层级整理结果，并输出关系判定状态与异常案例记录。

4.1.2 第一阶段：种子程序准备

第一阶段将原始程序筛选并整理为后续蜕变任务实例化可以直接使用的种子程序。进入后续流程的程序需要满足基本处理条件，即能够被稳定编译、执行、分析和改造。为此，种子程序进入种子池之前，统一执行表 4-1 所示的四类检查。

表 4-1 种子程序准备阶段的四类检查

检查类型	检查内容	目的
语法与编译检查	检查程序是否满足基本语法要求，能否在既定编译环境下成功编译，是否具备完整入口、必要声明与可解析结构	剔除语法不正确、无法编译或结构明显不完整的程序
运行可用性检查	检查程序能否稳定执行，是否依赖缺失的外部环境，是否包含明显未定义行为、难以复现的非确定性结构或异常终止路径	保证程序可被重复执行，并能够支撑后续插桩、观测与切片实验
工具链可处理性检查	检查程序能否被后续插桩、执行、切片和结果收集流程稳定处理，避免出现工具链无法接收或中途失效的样例	保证程序能够进入完整测试链
实验有效性检查	检查程序中是否存在后续可选观测对象、可利用的数据流或控制流结构，以及可支持程序改造与切片执行的局部上下文	保证程序不仅“能运行”，而且“值得测试”，能够为后续关系实例化与切片执行提供有效基础

在种子程序来源上，本文采用 Csmith 随机生成与大语言模型定向生成相结合的双轨构造方式。二者并不是简单并列的输入来源，而是分别服务于不同测试目标：Csmith 侧重扩大程序输入空间，覆盖常规 C 程序结构及工具前端处理路

径；大语言模型侧重根据测试目标构造具有明确语义特征的程序片段，以增强对复杂依赖场景和工具深层逻辑的触发能力。

对 Csmith 生成程序，第一阶段主要通过参数约束控制程序形态与后续处理难度。例如，可限制指针深度，并控制 bitfields 等会显著增加噪声或不利于后续处理的语言特性。经过这一处理后，种子程序在保持一定结构复杂度的同时，更便于后续的观测点筛选、程序改造和切片执行。Csmith 程序主要用于提供规模化、随机化的基础输入，以覆盖普通数据流、控制流和表达式组合场景。

对大语言模型生成程序，第一阶段重点关注其是否满足后续切片测试所需的语义特征和工程处理条件。本文采用“主题约束提示词—候选程序生成—编译检查—语义特征筛选—人工复核”的流程构造此类种子程序。具体而言，首先围绕指针别名、函数指针、回调传播、生命周期函数、隐藏状态和非常规控制流等高风险语义场景设计提示约束；随后由大语言模型生成候选 C 程序，并通过编译检查、运行检查和关系前提检查剔除语法错误、外部依赖过强、未定义行为风险较高或不满足测试目标的样例；最后将通过筛选的程序整理为统一种子程序，进入后续蜕变任务实例化流程。

通过上述双轨构造方式，第一阶段形成了覆盖型输入与定向型输入的互补。前者提高测试样例的数量和结构多样性，后者提高复杂语义场景的命中率；二者经过统一筛选后均被整理为相同形式的种子程序对象，从而避免后续任务实例化阶段受到输入来源差异的影响。表 4-2 总结了 Csmith 与大语言模型在种子程序构造中的作用差异。附录 B 详细介绍了大语言模型生成程序所使用的提示词模板和主题维度。

表 4-2 Csmith 与大语言模型在种子程序构造中的作用

生成方式	主要作用	典型覆盖场景	后续处理重点
Csmith 随机生成	扩大输入空间，提供规模化随机程序	普通数据流、控制流、表达式组合、工具前端处理路径	参数约束、编译检查、运行可用性检查
大语言模型定向生成	构造具有明确语义目标的测试程序	指针别名、函数指针、回调传播、生命周期函数、隐藏状态、非常规控制流	提示约束、语义特征筛选、关系前提检查与人工复核

第一阶段输出的是种子程序对象，而不是来源各异的原始程序文本。每个种子程序除程序本身外，还绑定最基本的描述信息，例如程序标识、程序来源、生成方式以及后续可追踪的版本信息，以支撑第二阶段的观测点筛选与任务实例化。

4.1.3 第二阶段：蜕变任务实例化

第二阶段围绕蜕变关系组织可执行、可比较的测试任务。该阶段主要处理候选观测点选择、改造点定位、派生程序或任务组构造，以及关系前提与比较方式的明确。表 4-3 给出了第二阶段的输入项与输出项。

表 4-3 第二阶段蜕变任务实例化的输入项与输出项

类别	项目	内容
输入	种子程序	通过第一阶段筛选的可用程序
	关系类型	当前任务对应的蜕变关系
	关系前提	当前关系成立所需满足的条件
输出	候选观测点集合	经筛选得到的候选变量或程序点
	派生程序或任务组	围绕关系语义构造得到的比较对象
	预期比较方式	后续关系判定所采用的比较规则
	蜕变测试任务	绑定上述信息的统一任务对象

第二阶段的任务实例化主要包括以下三个环节：

- **有效观测点选择：**第二阶段首先需要从种子程序中选出对后续切片测试真正有价值的候选观测点。对定向构造的样例，观测点通常可以直接指定；但对随机生成程序，尤其是 Csmith 生成的程序而言，全局变量和局部变量较多，需要根据变量是否参与有效计算、是否具有稳定输出以及是否能够被工具识别为切片准则进行筛选。
- **改造点选择与派生程序构造：**在确定观测点后，框架根据蜕变关系类型选择合适的改造位置，并生成相应的派生程序。对于语义保持关系，改造点通常位于准则相关计算路径上；对于无关数据流和无关控制流关系，改造点应避免改变原有准则相关行为，并保留新增成分的位置记录。对于多准

则组合一致性关系，则不需要构造派生程序，而是在同一程序点上组织单准则任务与组合准则任务。

- **关系前提检查：**派生程序或任务组生成后，需要检查其是否满足对应蜕变关系的前提条件，包括编译是否通过、运行是否稳定、观测值是否保持一致，以及新增成分是否符合数据无关或控制不可达等要求。只有通过前提检查的任务才进入后续切片执行与关系判定阶段。

4.1.4 第三阶段：切片准则选择与切片执行

候选观测点并不都适合作为切片准则。若所选变量未实际参与复杂数据流或控制流，对应切片往往只覆盖浅层局部结构，难以支撑有效比较。因此，本文优先从经过运行筛选且读写参与程度较高的候选变量中选择切片准则，并结合表 4-4 所示策略确定最终准则；随机选择仅作为补充，用于维持任务多样性。切片准则确定时不仅需要考虑变量本身，还需要结合其所在程序点、可观测性以及目标工具对准则形式的支持情况，以保证后续切片结果能够稳定提取和比较。

表 4-4 切片准则确定的三类策略

策略	具体做法	特点
随机选择	从候选观测点集合中随机选取变量作为切片准则	保持测试多样性，实现简单，但可能命中低价值变量
运行筛选后选择	先保留在程序实际执行中被使用的变量，再从中确定切片准则	排除无效观测点，减少无意义测试
读写统计排序后选择	根据变量的读写参与程度进行排序，再随机从前若干项中选取或直接选择排名最高者	更易命中复杂传播链上的变量，实验效率较高

切片准则确定后，框架根据待测工具的输入形式执行切片。对于源码级工具，任务直接围绕源程序中的目标程序点和变量集合展开；对于 LLVM IR 级工具，则先转换为中间表示，再在 IR 层级执行切片。对单观测点任务，切片围绕所选程序点和变量生成结果；对切片准则组合一致性关系，则在同一程序点上分别执行单准则任务与多准则任务，以保证结果可比较。第三阶段输出与测试任务、切片准则和工具标识绑定的原始切片结果，为后续整理与判定提供输入。

4.1.5 第四阶段：分层结果整理、关系判定与状态输出

不同工具的原始切片结果在表示形式和后续可利用信息上并不一致，因此这一阶段首先按工具特征对结果进行整理。对 Frama-C，直接整理其源码层切片结果；对 dg，在 LLVM IR / bitcode 层整理切片结果，并在该层级完成比较与判定。调试信息只用于异常位置定位和后续人工复核，不参与比较表示构造。

结果整理之后，框架依据蜕变关系类型执行判定。对语义保持类关系，判定的核心是相关程序在切片前后是否仍保持准则相关观测一致；对无关数据流和无关控制流关系，判定的核心是切片结果中是否仍保留理论上与准则无关的新增成分；对切片准则组合一致性关系，判定重点则转向多个切片结果之间是否满足预期集合关系。不同关系对应的判定对象和比较方式并不完全相同，因此第四阶段处理的不是统一的比较形式，而是在统一任务语境下选取相应的比较规则。

为统一后续实验分析口径，第四阶段的判定结果划分为三类：若切片执行成功且关系成立，则记为 PASS；若切片执行成功但关系被违背，则记为 VIOLATION；若由于工具失败、结果缺失、结果整理失败、关系前提失效或观测信息不足而无法形成可靠判定，则记为 ERROR。其中，PASS 表示当前任务未观察到异常，VIOLATION 表示当前任务产生了需要进一步分析的关系违背，ERROR 表示当前任务未能正常执行，可能是工具崩溃等问题，需要进一步分析。

第四阶段结束后，框架得到的是与测试任务绑定的统一状态结果及其比较依据。异常案例的进一步整理与输出见第 4.2 节。

4.2 框架输出

框架输出面向第四阶段判定后形成的异常案例。框架重点保留 VIOLATION 和 ERROR 两类结果，PASS 结果仅用于整体统计。表 4-5 给出了异常案例输出的主要记录项。

经过这一输出步骤，框架保留下来的不再是分散的工具原始结果，而是可直接进入后续分析的样例级异常对象。问题项级归并与当前状态整理不能由框架自动完成，而是建立在上述样例级结果和证据记录基础之上，由后续人工复核与统一编号过程完成。第五章将在这一基础上进一步开展问题项归并、类型划分和人工分析。

表 4-5 异常案例输出的主要记录项

异常类型	记录项
VIOLATION	测试任务标识；种子程序标识；关系类型；观测点；切片 准则；执行工具标识；整理后的结果对象；关系判定结 论；必要的前提检查信息
ERROR	测试任务标识；执行工具标识；失败位置；失败原因；必 要的上下文信息；是否进入可靠判定

4.3 本章小结

本章在第三章蜕变关系设计的基础上，给出了面向 C 程序切片工具的蜕变测试框架，并将验证流程组织为种子程序准备、蜕变任务实例化、切片准则选择与切片执行，以及分层结果整理、关系判定与状态输出四个阶段。

其中，种子程序准备阶段采用 Csmith 随机生成与大语言模型定向生成相结合的双轨构造方式，在输入空间扩展与复杂语义场景定向触发之间形成互补；蜕变任务实例化阶段围绕不同蜕变关系构造派生程序或任务组，并检查关系前提；切片执行与结果整理阶段则面向源码级和 IR 级工具的差异，分别组织结果收集、层级统一和关系判定。通过上述设计，框架将原始程序生成、切片任务构造、工具执行、结果整理和异常状态输出串联为一条可执行、可复核的验证流程，为第五章的实验统计与问题分析提供支撑。

第五章 实验设计与结果分析

5.1 实验配置与统计口径

5.1.1 实验对象与配置

本文选取源码级切片工具 Frama-C 与 LLVM IR 级切片工具 dg 作为实验对象，分别代表源码级分析路径与中间表示级分析路径。实验环境与主要工具配置如表 5-1 所示。

表 5-1 实验环境与工具配置

配置项	具体配置
操作系统	WSL2 环境下的 Ubuntu 22.04.5 LTS, x86_64
硬件环境	Intel(R) Core(TM) i5-14400F, 2.50 GHz; 内存 16.0 GB
C 编译环境	GCC 11.4.0
Frama-C	Frama-C 30.0 (Zinc)
dg	GitHub 源码构建版本
测试程序生成工具	Csmith 2.3.0; 大语言模型 GPT-5.4
覆盖率统计工具	Frama-C 使用 Bisect_ppx; dg 使用 gcov/gcovr

实验围绕第三章定义的四类蜕变关系展开，并基于第四章提出的测试框架组织测试任务。对每个进入执行流程的测试任务，本文记录其种子来源、关系类型、切片准则、工具输出、关系判定结果以及复核状态。对于检测到的异常样例，进一步进行去重、复测与人工复核，并将具有相同根因的异常样例归并为统一编号问题项。后文的总体结果、问题类型分布和代表性案例分析均建立在这一整理后的结果集之上。

5.1.2 统计口径与复核方式

为保证后文分析的一致性，本文区分“样例级异常”与“问题项级结果”两类统计对象。样例级异常用于描述具体测试任务中出现的关系违背现象，主要反映不同蜕变关系和不同种子来源的触发能力；问题项级结果则用于将具有相同

根因的异常样例进一步归并，作为总体结果、问题类型分布和代表性案例分析的统一单位。

此外，本文同时区分“问题状态统计口径”与“统一编号问题项统计口径”。前者用于说明问题的确认、提交与跟踪状态，后者用于支撑正文中的结构分析以及附录索引。除特别说明外，后文统计均以统一编号后的问题项为基本分析单位。

在复核阶段，本文主要依据异常是否可稳定复现、关系前提是否成立、切片结果差异是否具有明确语义解释以及是否能够归并到已有根因进行判断。对于边界样例，本文优先保守处理，避免将由样例构造不当或工具配置差异引起的现象直接归入工具问题。

5.2 总体结果与工具差异

本节首先从问题项总量及其当前归档状态出发，给出两类工具在当前实验中的总体结果；随后在统一功能视角下，对源码级与 IR 级切片工具的问题类型分布进行比较，以分析二者的共性风险与差异表现。

5.2.1 总体结果

在统一编号问题项整理完成后，本文进一步按照问题状态统计口径汇总两类工具的当前归档情况。总体上，Frama-C 共整理出 17 个问题项，dg 共整理出 26 个问题项，总计 43 个问题项。结合当前归档状态，Frama-C 中已有 13 个问题项经开发者完成确认，另有 4 个问题项已完成提交；dg 中已有 5 个问题项完成提交，其余 21 个问题项仍处于待提交状态。表 5-2 汇总了这一结果。

表 5-2 两种工具的问题状态分布

工具	已确认	已提交	待提交	问题项总数
Frama-C	13	4	0	17
dg	0	5	21	26

从总体规模看，两类工具均暴露出一批可以稳定复现的问题项，说明本文方法不仅能够触发异常，而且测试框架能够支撑后续复核、归档与外部跟踪。Frama-C 的问题项数量相对较少，但已有较高比例完成确认或提交；dg 的问题项

数量更多，当前则主要处于持续整理与逐步提交阶段。结合附录中的统一编号清单及其外部跟踪信息可以看到，这些结果已经形成了较清楚的证据链。

5.2.2 问题类型分布

为比较源码级与 IR 级切片工具在功能层面的共性失效特征，本文在总体分析中采用“欠保留、过保留、工具异常”三类统一口径。在附录 A.1 保留了每个统一编号问题项的原始记录信息，包括问题摘要、主归因关系和当前状态等内容；本节中的三类划分是在原始记录基础上，从功能后果角度对问题项作进一步抽象，以支持跨工具比较。附录承担问题项记录与追踪功能，本章则在统一功能视角下对其进行归并分析。

在这一统一口径下，欠保留是指切片结果未能正确保留准则相关语义，包括必要语句、依赖关系、控制语义、副作用或结果重建条件的遗漏与失真；过保留是指与切片准则无关的成分仍被错误保留；工具异常是指工具在分析、构图、切片或结果导出阶段未能稳定产生可接受切片产物。

其中，某些问题项在原始表现上可能体现为切片结果在编译、链接或运行阶段失效，但若其根因仍可归结为准则相关语义未被正确保留，则本文在统一口径下将其归入欠保留；只有当异常直接表现为切片工具自身在分析或导出过程中中止、崩溃或无法完成处理时，才归入工具异常。通过这一抽象，正文中的类型比较聚焦于功能失效核心问题，而不受源码级与 IR 级表示差异的干扰。

图 5-1 给出了两种工具在这一统一口径下的问题类型分布。两类工具的问题均以欠保留为主，其中 Frama-C 对应 9 项，dg 对应 19 项，表明准则相关语义未被完整保留是当前最主要的共性风险。过保留问题数量相对较少，Frama-C 与 dg 分别为 2 项和 1 项；工具异常分别为 6 项和 6 项。

从图 5-1 可以看出，两类工具的问题都以欠保留为主要失效形式，但其具体分布特征存在差异。Frama-C 的问题分布相对更为分散，除欠保留外，工具异常所占比例也较高。这表明源码级切片工具除语义保持不足外，还较容易在结果导出、重建与工程实现边界上暴露稳定性问题。dg 的问题则更集中于欠保留，说明其主要薄弱点仍位于依赖建模与语义传播过程，尤其体现在库副作用、隐藏状态、非常规控制流以及底层 IR 处理等路径上。

总体而言，两类工具都以准则相关语义未被正确保留作为主要风险，但

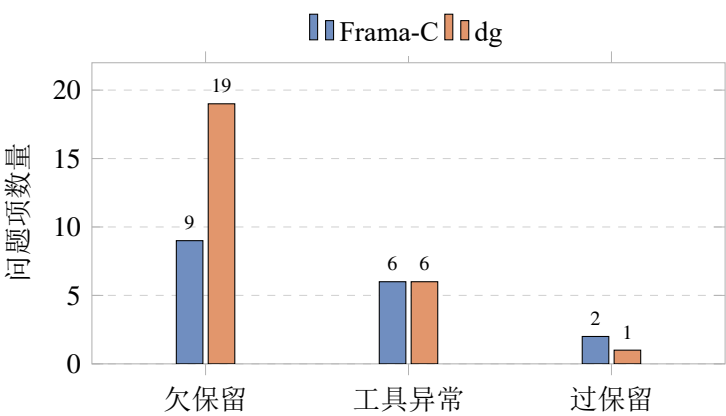


图 5-1 两种切片工具不同类型问题项数量分布

Frama-C 更容易同时暴露结果生成阶段的工程性问题，而 dg 的问题则更多体现为切片语义保持不足。这也说明，虽然源码级与 IR 级工具在外在表现上存在差异，但在统一功能视角下，“欠保留”是二者的核心失效模式。

5.3 各蜕变关系揭示的问题类型

表 5-3 给出了两种工具在统一编号问题项统计口径下按主归因蜕变关系划分的问题项分布。Frama-C 的 17 个问题项中，15 个归入 MR1，1 个归入 MR3，1 个归入 MR4；dg 的 26 个问题项中，25 个归入 MR1，1 个归入 MR3，未观察到由 MR2 或 MR4 主导的独立问题项。该结果表明，两类工具中的大多数问题首先表现为“准则相关”语义保持失败，而过保留与多准则一致性问题只对应少量更具体的独立问题项。

表 5-3 按主归因关系划分的问题项数量

工具	MR1	MR2	MR3	MR4
Frama-C	15	0	1	1
dg	25	0	1	0

5.3.1 MR1 主要揭示的问题类型

MR1 是两类工具中最主要的问题揭示关系。在 Frama-C 上，MR1 主导的问题主要包括源码导出后的初始化链断裂、符号重写失配、语言构造保真失败以及程序生命周期相关副作用遗漏。这些问题的共同特征是：切片结果在既定观测点上未能正确保持准则相关语义，因而最终表现为观测值偏差、程序运行失败或导

出结果不可用。在 **dg** 上，**MR1** 主导的问题则主要集中于外部语义效应建模不足和复杂控制流场景下的语义保持失败，包括库调用副作用遗漏、回调语义传播不完整、隐藏状态遗漏以及特殊 **IR/CFG** 场景下的结果失真。总体上，**MR1** 所揭示的是两类工具最核心的一组问题类型，即切片结果未能正确保持准则相关语义。这些问题在第 5.6.1 节、第 5.6.2 节以及第 5.6.4–5.6.6 节中有进一步展开。

5.3.2 MR2 主要揭示的问题类型

MR2 所检验的是切片结果能否有效剔除与准则无关的数据流成分。在本次实验中，**MR2** 未形成独立主归因问题项，但在 **Frama-C** 上揭示出一类稳定问题：部分与切片准则无关的数据流成分没有被完全消除。例如，对于某些全局变量，当其仅作为函数实参传入而未被实际使用时，相关调用语句已经在当前函数中被删除，但该变量的定义仍在最终切片结果中残留。

对相关样例的源码级分析发现，这类问题的根因不在于数据依赖分析未能识别无关实参与形参，而在于切片结果导出阶段对冗余成分的清理不彻底。具体而言，**Frama-C** 在生成最终切片结果时会对部分语句进行重写与整理；在这一过程中，虽然与准则无关的数据传播链已经被删除，但对应的全局定义或无实际作用声明仍被保留下来。因而，**MR2** 在 **Frama-C** 上揭示的问题类型可以概括为无关数据流残留与导出阶段冗余保留。

相比之下，在 **dg** 上，**MR2** 未观察到边界清晰且可独立归并的同类问题。这与 **dg** 在 **LLVM IR** 层面执行切片、且不直接面向源码导出有关。就本文实验结果而言，**MR2** 未将数据依赖隔离失败揭示为两类工具中的主要独立缺陷类型，说明在当前测试样例和判定原则下，两类工具对基本数据依赖与控制依赖的处理整体较为稳定。与此同时，部分样例中仍出现了与无关成分保留相关的异常现象，表明切片结果的精简性和结果后处理过程仍存在进一步改进空间。

5.3.3 MR3 主要揭示的问题类型

MR3 主要揭示控制不可达负载残留问题，即某些在当前控制条件下实际上不会执行的分支成分，仍被工具保留在最终切片结果中。实验表明，**Frama-C** 在简单控制不可达场景下表现较为完善。例如，对于 `if (a < 0 && a > 100)`

这类显然不可满足的逻辑矛盾条件，当变量 `a` 为普通变量时，相关分支能够被正确删除；这说明 **Frama-C** 在这类简单示例上具备较好的控制不可达性处理能力。相较之下，当 `a` 被声明为 `volatile` 时，相关成分仍会被保留，表明工具在涉及特殊语义约束时采取了更安全的处理策略。

Frama-C 在 **MR3** 场景下暴露出的代表性问题，集中体现为复杂程序中的控制不可达分支保留。具体而言，当某些分支在语义上实际上不会执行，但这一不可达性需要结合较复杂的数据流与控制流信息共同判断时，工具仍会基于保守性将该分支及其相关成分保留在切片结果中。本文在 **Csmith** 生成的复杂程序中稳定观察到这一问题，因此可将其概括为：复杂控制流与数据流交织场景下的控制不可达负载残留。

在 **dg** 上，**MR3** 进一步形成了一类独立问题项，其典型表现同样是控制不可达路径上的负载被稳定保留。总体而言，**MR3** 能够较稳定地揭示控制不可达负载残留问题，是识别过保留现象的有效蜕变关系。相关案例见第 5.6.6 节。

5.3.4 MR4 主要揭示的问题类型

MR4 所揭示的问题类型对应多准则切片场景下的一致性边界失稳。在 **Frama-C** 上，**MR4** 对应一个独立问题项，其表现为多变量切片结果相较于单变量切片结果的合理组合额外扩张，保留了与准则无关的全局变量。这说明相关问题发生在多准则合并策略这一更具体的层面。在当前样例分布与判定原则下，**dg** 上尚未观察到边界清晰且稳定的 **MR4** 独立问题项。就结果层面看，**MR4** 主要揭示的是多准则组合下一致性边界失稳问题。相关案例见第 5.6.3 节。

综合来看，**MR1** 主要对应准则相关语义保持失败，**MR2** 对应无关数据流残留，**MR3** 对应控制不可达负载残留，**MR4** 对应多准则组合下一致性边界失稳。四类关系在本文中的区别主要体现在它们分别把问题组织为不同类型的事实。表 5-4 对这一对应关系作进一步概括。

5.4 不同种子来源的贡献

本节比较 **Csmith** 与大语言模型生成测试用例在问题发现中的作用差异。这里采用统一编号问题项统计口径：**Frama-C** 按正文统一编号的 17 个问题项统计，

表 5-4 四类关系揭示的问题类型与对应讨论位置

MR	主要问题类型	在本文中的主要表现	对应讨论位置
MR1	准则相关语义保持失败	初始化链断裂、符号重写失配、语言构造保真失败、内存副作用遗漏、回调与隐藏状态传播遗漏、非常规控制流语义遗漏	第 5.6.1 节、第 5.6.2 节、第 5.6.4–5.6.6 节
MR2	无关数据流残留	无关全局状态、辅助赋值链或冗余声明残留	第 5.3.2 节
MR3	控制不可达负载残留	逻辑矛盾分支、零迭代循环、非选中 switch 或死路径成分仍被保留	第 5.3.3 节、第 5.6.6 节
MR4	多准则一致性边界失稳	多准则结果相较单准则合理组合额外扩张,组合后无关变量残留	第 5.6.3 节

dg 按正文统一编号的 26 个问题项统计。表 5-5 给出了两类工具在不同种子来源下对应的问题项数量。

表 5-5 不同种子来源揭示的问题项数量

工具	Csmith	LLM
Frama-C	10	7
dg	1	25

在 Frama-C 上, Csmith 与大语言模型生成程序分别揭示了两类不同问题。Csmith 主要发现的是复杂源码结构下的切片错误, 例如初始化传播缺失、局部常量数组初始化丢失、指针初始化被删除后发生解引用错误、除数相关初始化缺失, 以及一般性的过保留。这些问题的共同特点是: 程序本身不依赖特殊语言机制, 但包含较长的数据流链、初始化链或较复杂的控制传播, 因此更容易暴露工具在依赖恢复和源码导出上的基本缺陷。

相比之下, 大语言模型生成程序在 Frama-C 上主要发现语义边界场景中的问题, 包括函数重命名后函数指针仍指向旧符号、va_list 被生成为非法数组类型赋值、切片结果保留 __fc_vla_alloc 但缺少可链接实现、以及 constructor、destructor 和 atexit 相关生命周期副作用遗漏等。这些问题通常需要围绕函数指针、变参、运行时辅助符号和生命周期语义进行专门构造后才更容易暴露。就结果而言, Csmith 更容易发现复杂普通代码中的依赖恢复问题, 而大语言模型生成程序更容易发现编程语言边界和运行时语义场景中的处理缺陷。

在 `dg` 上，两类种子来源的差异更为明显。按本节统计口径，归入大语言模型生成程序的共有 25 个问题项。它们主要包括库调用副作用未被正确建模、回调和隐藏状态传播缺失、非局部控制转移处理不当，以及若干特殊 IR/CFG 场景下的切片错误和内部异常。换言之，大语言模型生成程序在 `dg` 上主要揭示的是复杂 C 语义和特殊 IR 场景中的核心问题。

相比之下，`Csmith` 在 `dg` 上只补充发现了一个新的底层问题，即 `ConstantExpr` 指针比较及其相关全局指针结构处理会在构图阶段触发崩溃。这与两个工具的支持程度有关，因为 `dg` 当前对复杂程序的支持能力有限，对于结构较复杂的随机程序往往难以稳定完成切片，同时整体切片效率也较低，难以将大规模随机样例作为主要测试来源。在这一条件下，大语言模型生成程序更适合围绕可控语义场景构造可执行样例，从而承担 `dg` 上多数问题的揭示工作；而 `Csmith` 仍然保留补充发现底层 IR 与指针处理路径特殊缺陷的价值。

综合来看，`Csmith` 与大语言模型生成程序在本文中形成了互补关系。在 `Frama-C` 上，前者主要发现复杂不涉及特殊语义机制程序中的依赖恢复与结果导出问题，后者主要发现函数指针、变参、生命周期函数以及运行时辅助符号等语言边界场景中的处理缺陷；在 `dg` 上，多数问题由大语言模型生成程序揭示，而 `Csmith` 补充底层 IR 与指针处理上的个别新问题。因此，根据实验结果，二者分别对应不同的问题类型，并在不同工具条件下承担不同的发现作用。

与 `Cuoq` 等人基于 `Csmith` 的 `Frama-C` 随机测试工作相比^[71]，本文实验在测试预言和问题类型上具有一定差异。`Cuoq` 等人的工作主要通过比较原程序与切片后程序的 `checksum` 是否一致来构造测试预言，因此更适合发现切片后程序行为发生改变的健全性问题，但难以发现无关代码被过度保留或关系不一致等问题。本文采用领域专用蜕变关系作为判定依据，除关注语义保持失败外，还能够揭示无关数据流残留、不可达控制负载残留以及多准则组合不一致等问题。因此，本文方法在测试预言形式和可揭示问题类型上效率更高，成本更低。

5.5 代码覆盖率分析

本节从工具实现触达范围的角度考察四类蜕变关系测试集的覆盖效果。由于两类工具的底层开发语言不同，`Frama-C` 基于 `OCaml` 生态，`dg` 基于 `C/C++` 编

译工具链，因此其覆盖率信息统计特征并不相同：前者依赖 Bisect_ppx 检测得到覆盖信息，后者依赖 gcov/gcovr 生成行覆盖率与分支覆盖率统计。基于这一差异，本文主要比较同一工具内部不同蜕变关系测试集的相对触达差异。

5.5.1 Frama-C：总体覆盖率与核心模块穿透

从覆盖率信息看，Frama-C 在四类蜕变关系测试集下的覆盖率数据分别为：MR1 的 20.88%、MR2 的 18.19%、MR3 的 18.22% 和 MR4 的 19.81%。这说明四类关系都能够稳定进入工具内部的有效执行区域，同时为了解释测试究竟触达了哪些关键分析模块，本文进一步围绕前端解析、PDG 构建、Eva 值分析与核心切片逻辑四组关键模块进行聚合。表 5-6 给出了这些模块组在四类关系中取得的最高覆盖率。

表 5-6 Frama-C 核心模块最高覆盖率

模块组	主要职责	对应关系	最高覆盖率
前端解析	语法解析与词法分析	MR1	25.6%
PDG 构建	依赖图构建与控制依赖传播	MR1	80.0%
Eva 引擎	值分析与抽象状态求值	MR1	57.0%
核心切片	切片计算与标记传播	MR1	59.7%

图 5-2 展示了四类蜕变关系在这些核心模块组上的覆盖率分布。可以看到，MR1 在四组关键模块上均达到最高值，尤其是在前端解析与 PDG 构建上优势最为明显。这说明语义等价变换不仅保持了程序行为约束，还通过重写表达式结构、控制流形态和调用形式，迫使 Frama-C 的解析与依赖构建链路处理更广泛的源码模式，因此更容易形成较深的实现触达。

相比之下，MR2 与 MR3 在四组模块上的覆盖分布几乎一致。这一结果与两类关系的设计目的相符：无关数据流扰动与无关控制流扰动都更强调对既有切片任务的局部干预，因此虽然能够稳定进入核心分析路径，但对前端和构图阶段的结构多样性扩展相对有限。换言之，二者的作用更偏向于“在已进入的分析链路上施加针对性压力”，而不是显著扩大被访问的实现区域。相比之下，MR4 在 PDG 构建与核心切片模块上的覆盖略高于 MR2/MR3，说明多准则组合关系确实能够有效触达结果合并、标记传播与依赖回溯等关键逻辑。

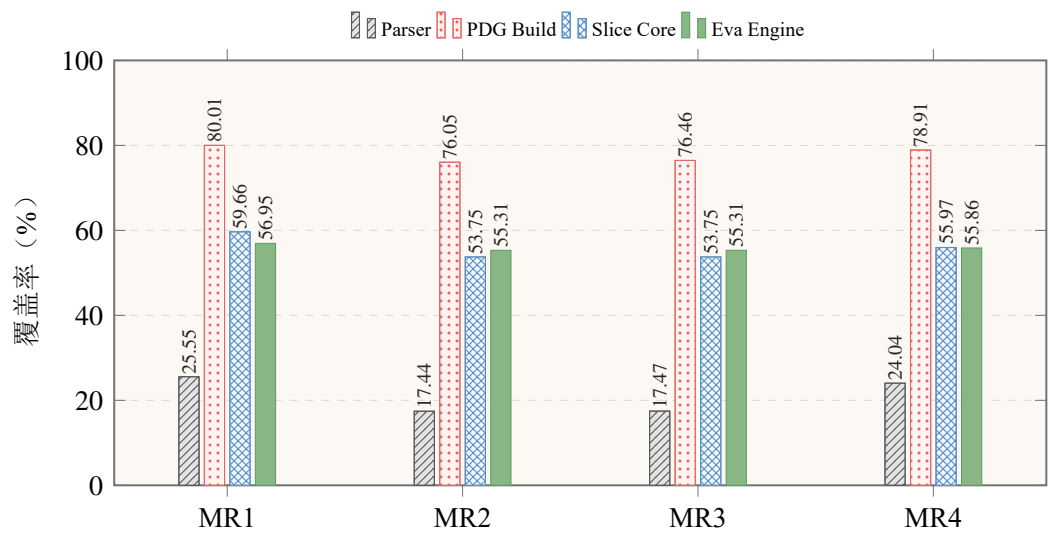


图 5-2 *Frama-C* 四类蜕变关系测试在四组核心模块上的覆盖率对比

5.5.2 dg: 总体覆盖率与核心模块分布

从总体覆盖率看，按行覆盖率与分支覆盖率取平均后，*dg* 在四类蜕变关系测试集下的平均覆盖率分别为：MR1 的 45.0%、MR2 的 36.2%、MR3 的 37.3% 和 MR4 的 38.7%。这说明四类关系都能够稳定进入工具内部的有效执行区域，其中 MR1 的总体触达能力最强。为进一步解释测试究竟触达了哪些关键实现模块，本文围绕 *include*、*lib* 与 *tools* 三组核心模块进行聚合。表 5-7 给出了这些模块组在四类关系中取得的最高覆盖率。

表 5-7 *dg* 核心模块最高覆盖率

模块组	主要职责	对应关系	最 高 覆 盖 率 (行 / 分支)
include	工具接口声明、公共数据结构以及配置相关头文件，支撑切片入口与分析对象组织	MR1	59.2% / 37.8%
lib	依赖图构建、语义传播、切片计算等核心分析库实现	MR1	44.5% / 37.3%
tools	切片驱动、参数解析、任务调度与结果输出等工具层实现逻辑	MR1	60.5% / 46.5%

图 5-3 进一步给出了四类蜕变关系测试的覆盖率分布。可以看到，MR1 的行覆盖率和分支覆盖率均为最高；MR4 在其余三类关系中相对更高；MR2 与 MR3 则保持在中间水平。这一分布与前文缺陷结果相互印证，即 MR1 仍然是最能驱动 *dg* 进入语义脆弱区的主导关系，而 MR4 的组合准则场景则在统一测试框架内提供了额外的覆盖增量。

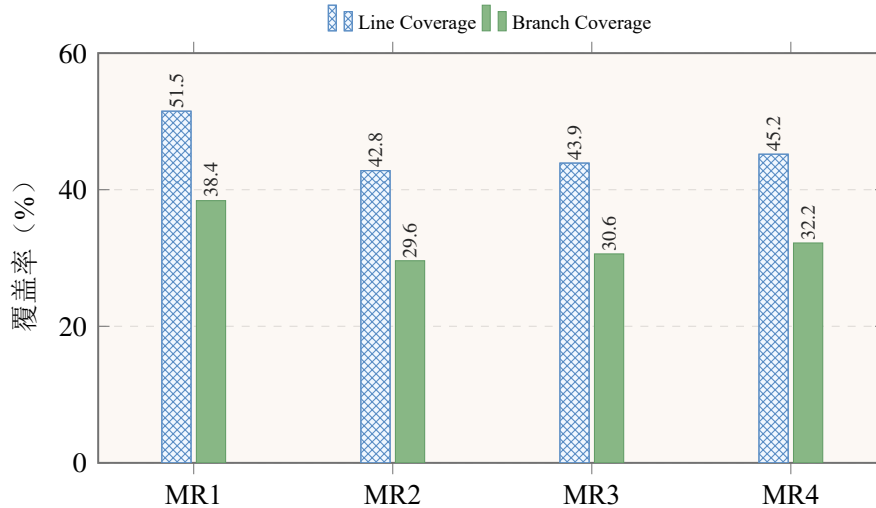


图 5-3 dg 四类蜕变关系测试的行覆盖率与分支覆盖率对比

结合表 5-7 可以看到，MR1 在三组核心模块上均取得最高覆盖率。其中，include 与 tools 的行覆盖率均在 60% 左右，同时 tools 的分支覆盖率达到到了 46.5%，说明 MR1 不仅能够较充分地触达工具入口、接口组织与配置驱动路径，也能够稳定推进到切片执行、任务调度与结果输出等关键实现区域。相比之下，lib 模块组的最高行覆盖率与分支覆盖率分别为 44.5% 和 37.3%，虽然同样表明测试已经进入依赖分析与切片计算所在的核心库层，但其内部仍有一部分实现路径未被当前测试样例充分覆盖。

总体而言，dg 的覆盖率结果与 Frama-C 呈现出相似的分布特征：在蜕变关系设计层面，四类关系都能够稳定进入工具内部的有效执行区域，其中 MR1 的触达能力最强；在核心模块层面，测试已经能够稳定进入 include、lib 与 tools 所代表的关键区域，并对工具的接口组织、分析执行与切片实现路径施加有效压力。

5.6 典型缺陷根因与代表性案例

本节从缺陷根因出发，对已识别问题项进行归纳分析。本文选取 6 类具有代表性的失效模式展开讨论，其中 Frama-C 与 dg 各保留 3 类案例。Frama-C 的问题主要体现为源码级结果导出、自治性保持和多准则处理边界问题；dg 的问题则主要集中在内存副作用建模、回调与隐藏状态传播、非常规控制流及 IR 层健壮性边界等方面。

5.6.1 Frama-C：依赖链恢复与源码导出自洽性破坏

这一类问题的共同特征是：切片结果保留了后续使用链条，却在源码导出阶段切掉了支撑该使用的初始化语句、声明初始化值或中间传播链。其根因通常不在于切片任务无法执行，而在于切片结果被重建为源码程序时，定义端与使用端之间的自洽关系被破坏。

案例 1：局部初始化链丢失

```

1  static int32_t g_34 = 0x9EED08E1L;
2  static int32_t g_1320 = 4L;
3
4  static void func_1(void)
5  {
6      int32_t *l_36 = &g_34;
7      int32_t **l_35 = &l_36;
8      int32_t * const l_2338 = &g_1320;
9      (**l_35) |= (*l_2338);
10 }
```

例 5.1(a) defect 4 的原程序关键片段

```

1  static int32_t g_34 = (int32_t)0x9EED08E1L;
2
3  static void func_1_slice_1(void)
4  {
5      int32_t *l_36 = &g_34;
6      int32_t **l_35 = &l_36;
7      {
8          int32_t * const l_2338; /* 初始化已丢失 */
9          *(*l_35) |= *l_2338;
10     }
11 }
```

例 5.1(b) defect 4 的切片结果关键片段（注释为本文后加）

上述片段中，原程序通过 `l_2338 = &g_1320` 建立对全局变量 `g_1320` 的引用，并在后续表达式中解引用该指针。切片结果保留了对 `*l_2338` 的使用，却删除了 `l_2338` 的初始化语句，导致切片后程序中的依赖链断裂。统一编号 8、9 和 10 与该问题属于同类变体，其共同根因都是源码级切片结果保留了

使用点，却在导出阶段破坏了支撑该使用点的初始化链条。

5.6.2 Frama-C：函数指针与语言边界构造处理不一致

这一类问题主要发生在函数指针、变参、运行时辅助符号等语言边界场景中。与普通依赖链丢失不同，这类问题更强调源码级导出结果是否仍满足 C 语言本身的符号引用、调用约定和语言构造使用规则。

案例 2：函数指针重命名不同步

```

1  typedef int (*func_ptr_t)(int, int *);
2
3  int add_func(int x, int *p) {
4      *p += x;
5      return *p;
6  }
7
8  int mul_func(int x, int *p) {
9      *p *= x;
10     return *p;
11 }
12
13 func_ptr_t get_func(int choice) {
14     if (choice == 1) return add_func;
15     if (choice == 2) return mul_func;
16     return add_func;
17 }

```

例 5.2(a) defect 5 的原程序关键片段

在该问题中，切片结果已经生成了新的函数定义 `add_func_slice_1` 和 `mul_func_slice_1`，但函数指针返回路径仍然引用旧符号 `add_func` 和 `mul_func`。这说明 Frama-C 在源码导出阶段只完成了定义端的重写，而未同步更新函数指针引用端。一旦相关旧符号未被完整保留，切片结果就可能在链接阶段失败。类似地，变参对象 `va_list` 被错误改写为普通指针操作、运行时辅助符号被保留但缺少可链接实现等问题，也可归入语言边界构造处理不一致这一类。

```

1  int add_func(int x, int *p);
2  int add_func_slice_1(int x, int *p){...}
3  int mul_func(int x, int *p);
4  int mul_func_slice_1(int x, int *p){...}
5
6  func_ptr_t get_func_slice_1(int choice)
7  {
8      func_ptr_t __retres;
9      if (choice == 1) {
10         __retres = & add_func; /* 仍引用旧符号 */
11         goto return_label;
12     }
13     __retres = & mul_func; /* 仍引用旧符号 */
14     return_label: return __retres;
15 }

```

例 5.2(b) defect 5 的切片结果关键片段（注释为本文后加）

5.6.3 Frama-C：多准则组合一致性问题

这一类问题对应 MR4 所检验的多准则组合一致性。其核心现象是：组合准则的切片结果超出了相应单准则切片结果的并集边界，额外保留了与组合准则无关的程序成分。

案例 3：多准则组合下的并集外无关全局变量卷入

```

1  static int g_noise = 0;
2  static int g_keep_a = 0;
3  static int g_keep_b = 0;
4  static int pick_a(int x, int noise) { return x; }
5  static int pick_b(int noise, int y) { return y; }
6
7  int main(void) {
8      g_keep_a = pick_a(1, g_noise++);
9      g_keep_b = pick_b(g_noise++, 2);
10     return g_keep_a + g_keep_b;
11 }

```

例 5.3(a) 按 defect 12 原程序关键片段

```

1  static int g_noise = 0;  /* 组合切片中额外保留的无关全局变量 */
2  static int g_keep_a = 0;
3  static int g_keep_b = 0;
4
5  static int pick_a_slice_1(int x) { return x; }
6  static int pick_b_slice_1(int y) { return y; }
7
8  int main(void)
9  {
10     g_keep_a = pick_a_slice_1(1);
11     g_keep_b = pick_b_slice_1(2);
12     return g_keep_a + g_keep_b;
13 }

```

例 5.3(b) 组合准则 {g_keep_a, g_keep_b} 的切片结果（注释为本文后加）

在该模式下，分别以 g_keep_a 和 g_keep_b 作为单准则执行切片时，g_noise 均应被删除，因此两个单准则结果的并集中不应包含该全局变量。若组合准则切片结果保留 g_noise，则说明多准则切片结果超出了单准则结果并集边界。该问题反映的是多准则合并与结果导出过程中的一致性边界失稳，而不是单一准则下的普通无关数据流残留。

5.6.4 dg：内存对象写入效应遗漏

这一类问题的共同特征是：后续断言或观测语句依赖某个内存对象在调用后的更新状态，但切片结果没有同时保留产生该状态更新的调用或其前置依赖。问题根源在于工具未能将“调用对目标对象的写入效应”及其前置状态形成过程稳定纳入依赖图。

案例 4：保留 memmove，却切掉其源依赖

如例 5.4(a) 和例 5.4(b) 所示，断言依赖的是 buf 在两次内存写入之后形成的最终状态。原始 IR 中同时存在两条 llvm.memcpy 和一条 llvm.memmove，而切片后 IR 仅保留了 llvm.memmove，删除了为其提供源对象状态的前置 llvm.memcpy。这说明 dg 在处理内存对象相关调用时，仅保留了距离断言更近的最终写入动作，却没有继续追溯该写入动作所依赖的源内存状态。对于

memmove 而言，其语义不仅取决于目标地址和拷贝长度，也取决于源地址所指向内存区域在调用前已经形成的内容。因此，若前置 memcpy 被删除，切片结果虽然保留了表面上的最终写入调用，但已经破坏了该调用的语义来源，导致断言所依赖的内存状态无法被正确重建。统一编号 19、24、25、26、28、29、30 和 31 与其共享同一失效模式，即后续观测点依赖的是内存对象上的状态变化，而不是某个调用的返回值。

```

1  #include <assert.h>
2  #include <string.h>
3
4  int main(void) {
5      char src[] = "hello";
6      char buf[16];
7      if (1) {
8          memcpy(buf, src, 6);
9          memmove(buf + 2, buf, 3);
10     }
11     int probe_ok = (buf[2] == 'h' && buf[3] == 'e' && buf[4] == 'l');
12     assert(probe_ok);
13     return 0;
14 }

```

例 5.4(a) defect 18 原程序关键片段

```

1  ; original.ll
2  call void @llvm.memcpy.p0i8.p0i8.i64(...), !dbg !21
3  call void @llvm.memcpy.p0i8.p0i8.i64(...), !dbg !27
4  call void @llvm.memmove.p0i8.p0i8.i64(...), !dbg !32
5
6  ; sliced.ll
7  call void @llvm.memmove.p0i8.p0i8.i64(...), !dbg !26
8  ; 切片后模块中丢失 llvm.memcpy

```

例 5.4(b) defect 18 的原始 IR 与切片后 IR 关键摘录

5.6.5 dg：回调与隐藏状态传播遗漏

这一类问题的特点是：程序的可观察行为依赖库调用过程中经由回调函数或隐藏状态传播形成的结果。若切片仅围绕显式返回值或局部 SSA 使用关系组

织依赖，就可能遗漏这类非局部语义路径。

案例 5：回调驱动的状态变换被删除

```

1  #include <assert.h>
2  #include <stdlib.h>
3
4  static int cmp(const void *a, const void *b) {
5      int x = *(const int *)a;
6      int y = *(const int *)b;
7      return (x > y) - (x < y);
8  }
9
10 int main(void) {
11     int a[3] = {3, 1, 2};
12     qsort(a, 3, sizeof(int), cmp);
13     assert(a[0] == 1 && a[1] == 2 && a[2] == 3);
14     return 0;
15 }

```

例 5.5(a) defect 27 的触发程序

```

1  ; original.ll
2  %5 = bitcast [3 x i32]* %2 to i8*
3  call void @qsort(i8* noundef %5, i64 noundef 3, i64 noundef 4,
4                  i32 (i8*, i8*)* noundef @cmp), !dbg !25
5
6  ; sliced.ll
7  ; 切片后 IR 中未保留 @qsort 调用

```

例 5.5(b) defect 27 的原始 IR 与切片后 IR 关键差异

在该案例中，断言依赖 `qsort` 调用和比较函数 `cmp` 形成的数组重排状态。切片后 IR 中 `@qsort` 调用被删除，说明 `dg` 未能将“库接口调用—用户回调函数—数组状态变化”语义链纳入依赖图。与普通函数调用不同，`qsort` 的影响不体现在返回值上，而是通过回调比较函数决定元素顺序，并原地修改数组状态。若工具只追踪显式局部变量赋值，缺少库调用副作用和回调关系建模，容易遗漏跨调用边界依赖。类似地，`strtoll` 对 `errno` 的隐藏状态更新、环境变量、流状态和伪随机状态等，也可能因缺少显式局部值流而被遗漏。这类问题表明，`dg` 对库调用、回调函数和隐藏状态形成的非局部语义依赖建模仍不充分。

5.6.6 dg: 非常规控制流与 IR/CFG 健壮性边界

这一类问题主要涉及普通 CFG 难以直接表达的控制语义，或 LLVM IR/CFG 层面的特殊结构。当前者未被正确建模时，切片结果会遗漏决定执行路径的关键控制依赖；当后者超出工具处理边界时，则可能直接表现为构图失败、分析中止或内部异常。

案例 6：非局部控制流依赖遗漏

```

1  #include <assert.h>
2  #include <setjmp.h>
3
4  static jmp_buf jb;
5
6  static void jumpit(void) {
7      longjmp(jb, 1);
8  }
9
10 int main(void) {
11     volatile int x = 0;
12     if (setjmp(jb) == 0) {
13         x = 1;
14         jumpit();
15         x = 2;
16     }
17     assert(x == 1);
18     return 0;
19 }

```

例 5.6(a) defect 20 的真实触发程序

```

1  ; original.ll
2  %3 = call i32 @_setjmp(...), !dbg !48
3  call void @longjmp(...), !dbg !65
4
5  ; sliced.ll
6  %2 = call i32 @_setjmp(...), !dbg !48
7  ; 切片后 IR 中未保留 @longjmp 调用

```

例 5.6(b) defect 20 的原始 IR 与切片后 IR 关键摘录

在这一例中，断言成立依赖 `setjmp/longjmp` 共同形成的非局部返回路径。原始 IR 同时包含 `@_setjmp` 与 `@longjmp`，但切片后 IR 删除了真正触发控制回跳的 `@longjmp` 调用，导致非常规控制语义没有被完整保留。除这类非局部控制流遗漏外，`dg` 还在大规模 `switch`、间接跳转、向量元素插入、COMDAT/C++ 相关结构、调试元数据以及 `ConstantExpr` 指针表达式等场景中暴露出内部健壮性边界。这说明 `dg` 的薄弱点不仅体现在依赖建模不足，也体现在部分特殊 IR/CFG 结构的工程处理边界上。

5.7 有效性威胁

尽管本文围绕典型 C 语言切片工具、真实程序样例和多类蜕变关系开展了较系统的实验，但所得结论仍然受到若干有效性因素的约束。为避免对实验结果作超出证据范围的解释，本节从构念有效性、内部有效性、结论有效性和外部有效性四个方面讨论本文结论的边界。

5.7.1 构念有效性

本文以蜕变关系违背、分层结果整理后的切片结果比较、问题项级归并以及人工复核结果作为主要观测对象。这一构念能够较好支撑“程序切片工具是否在当前测试场景下表现异常”的判断，但仍存在不足，例如 MR1 的“语义保持”在本文中通过统一观测点下的观测值一致性来代理，而不是通过完整语义等价证明来保证；MR2 与 MR3 也分别通过新增成分是否残留来代理数据依赖隔离与控制可达性排除能力。因此，本文更适合被理解作为一种围绕切片工具关键边界开展的关系驱动验证，而不是对程序切片完整语义正确性的穷尽检验。

5.7.2 内部有效性

内部有效性主要受到样例构造质量、关系前提成立性、结果整理实现细节和人工复核过程的影响。首先，若某些派生程序本身未能真正满足蜕变关系前提，则由此产生的异常不能完全归因于切片工具缺陷；为降低这一风险，本文在关系实例化与案例复核中尽量保留了关系前提检查与观测验证步骤。其次，Frama-C 与 `dg` 的切片结果分别经过分层结果整理后，才用于各自工具内部的关系判定与

结果分析：若结果整理实现本身存在噪声，也可能影响后续异常识别与问题归并。再次，问题项级归并与当前状态整理中仍包含人工判断成分，因此不同研究者在边界样例归类上可能存在细微差异。本文通过统一编号表、关系主归因规则与附录案例索引来减轻这一影响，但无法完全消除人工解释因素。

5.7.3 结论有效性

本文的统计结果建立在当前样例规模、当前工具版本、当前问题归并方式以及当前复核口径之上，因此不能将其直接外推为对所有程序切片工具、所有蜕变关系或所有输入来源都成立的普遍结论。尤其需要说明的是，问题状态统计口径与统一编号问题项统计口径服务的目标不同：前者反映问题的外部跟踪进度，后者反映论文内部的分析单位，二者不能混用。与此同时，覆盖率结果可以为测试充分性提供证据，但不能直接替代问题揭示能力的比较，因此不能仅依据某一关系测试集覆盖率更高，就将其解释为“绝对更有效”。总体而言，本文的结论更适理解解为：在当前实验条件下，哪些关系、哪些输入来源和哪些程序模式更容易揭示两类工具的稳定薄弱点。

此外，由于 Csmith 随机生成程序与大语言模型生成程序在不同工具上的使用比例和任务规模并不完全一致，本文未对两类测试程序来源的覆盖率进行严格横向比较，而主要从蜕变关系和工具核心模块角度分析覆盖情况。

5.7.4 外部有效性

本文选取的对象为 Frama-C 与 dg，它们分别代表源码级与 LLVM IR 级切片实现路径，但仍不能覆盖所有程序切片工具、所有语言子集以及所有分析配置与工程环境。本文关注的语言对象主要是可由当前工具链稳定处理的 C 程序，因此对更广泛的 GNU 扩展、内联汇编、大规模系统代码以及复杂构建条件下的程序覆盖仍然有限。

同时，本文采用的输入来源为 Csmith、大语言模型生成程序以及围绕语义保持关系构造的派生程序。这些来源虽然能够形成互补，但仍不能等同于真实工业项目中的完整程序分布。尤其是工业代码中常见的构建脚本依赖、第三方库约束、宏配置差异和跨模块交互复杂性，在本文实验中尚未得到充分覆盖。

因此，本文更适合被理解为一项面向真实工具的实证研究：它说明所提方法在当前条件下能够发现何种问题、主要揭示哪些实现边界，而不应被直接外推为对更广泛程序切片工具、程序分布或工程场景都成立的一般性结论。

5.8 本章小结

本章围绕面向 C 程序切片工具的蜕变测试方法开展实验设计与结果分析。首先，说明了实验对象、工具配置、测试输入来源、统计口径与问题复核方式，为后续结果统计提供统一基础。随后，分别从总体问题数量、工具差异、问题类型分布和蜕变关系贡献等角度，对 Frama-C 与 dg 上发现的问题进行了统计分析。实验共整理出 43 个可复现问题项，其中 13 个问题已被相关工具开发者确认。

实验结果表明，本文设计的四类蜕变关系能够在两类不同实现层级的切片工具上产生有效判定信号。其中，语义保持关系主要用于观察相关路径改写后的切片稳定性，无关数据流与无关控制流关系主要用于揭示过度保留和可达性处理问题，多准则组合关系则用于检验组合切片任务中的结果一致性。不同测试程序来源在问题发现上也表现出差异：Csmith 程序更适合覆盖常规随机程序结构，大语言模型生成程序则更容易构造指针、回调、生命周期函数和非常规控制流等定向语义场景。

此外，本章还从代码覆盖率角度分析了测试任务对待测工具核心模块的触达情况，并通过代表性案例归纳了依赖链恢复、特殊语言构造处理、生命周期语义建模、多准则组合一致性、不可达控制路径残留以及内存副作用传播等典型问题。最后，本章从构念有效性、内部有效性、结论有效性和外部有效性四个方面讨论了实验结论可能受到的影响。上述分析为下一章总结全文工作、回应研究问题并讨论后续改进方向提供了实验依据。

第六章 总结与展望

6.1 研究总结

围绕绪论中提出的四个研究问题，本文结合第五章的实验结果，对所提方法的有效性、不同蜕变关系的作用差异、不同测试程序来源的贡献以及对工具关键实现路径的触达情况总结如下。

RQ1：所提方法能否在真实 C 语言程序切片工具上发现可复现的功能性失效？

实验结果表明，本文提出的蜕变测试方法能够在真实 C 程序切片工具上发现可复现、可复核的问题项。在统一编号问题项统计口径下，本文共整理出 43 个问题项，其中 Frama-C 为 17 个，dg 为 26 个。在问题状态统计口径下，Frama-C 中已有 13 个问题项经开发者确认，另有 4 个问题项已完成提交；dg 中已有 5 个问题项完成提交，其余问题项仍处于整理与提交阶段。这说明，在缺乏标准切片答案的情况下，将切片语义约束转化为蜕变关系判定，能够为 C 程序切片工具的自动化缺陷检测提供有效依据。

RQ2：不同蜕变关系在缺陷揭示中的作用是否存在差异？

实验结果表明，四类蜕变关系在问题揭示中具有不同侧重。MR1 主要揭示准则相关语义保持失败，是当前实验中贡献最多的问题来源；MR2 虽未形成独立主归因问题项，但能够辅助观察无关数据流残留和导出阶段冗余保留现象；MR3 主要揭示控制不可达负载残留问题；MR4 则用于检验多准则切片下的组合一致性边界。由此可见，四类关系分别从语义保持、数据非干扰、控制非干扰和组合一致性角度刻画切片工具的潜在风险，形成了互补的检验方式。

RQ3：大语言模型引导生成与传统随机生成在不同工具和缺陷类型上分别发挥何种作用？

实验结果表明，Csmith 随机生成程序与大语言模型引导生成程序在问题发现上具有互补性。对 Frama-C 而言，Csmith 更容易揭示复杂普通程序结构下的

依赖恢复、初始化链保留和结果导出问题，大语言模型生成程序则更容易触发函数指针、变参、生命周期函数和运行时辅助符号等语言边界场景中的处理缺陷。对 `dg` 而言，大语言模型生成程序在当前样例集中揭示了更多与库调用副作用、隐藏状态、回调传播、非局部控制转移和特殊 IR/CFG 处理相关的问题，而 Csmith 仍能够补充暴露底层 IR 与指针处理路径上的个别脆弱点。

RQ4: 所提方法是否能够更充分地进入切片工具的关键实现路径，并揭示不同实现层级上的薄弱点？

覆盖率分析和代表性案例表明，本文测试集能够触达切片工具的关键实现区域。对 Frama-C 而言，测试能够进入前端解析、PDG 构建、Eva 值分析和核心切片逻辑等模块，暴露的问题主要集中在源码级结果导出、语言构造保真以及依赖保留边界上；对 `dg` 而言，测试能够进入接口组织、依赖图构建、切片执行和结果输出等核心路径，暴露的问题更多集中在内存副作用建模、隐藏状态传播、非常规控制流处理以及 LLVM IR 健壮性上。这说明，源码级工具与 IR 级工具虽然都服务于程序切片任务，但其失效机制和工程薄弱点具有明显差异。

6.2 研究不足

尽管本文在真实工具上获得了一组可复核的实验结果，但研究仍然存在若干限制。

首先，本文对 MR1 的验证建立在统一观测点上的观测值一致性这一可执行代理条件之上，而不是完整切片语义等价的形式化证明。该处理方式能够支撑自动化实验，并与当前框架中的插桩、比较和复核流程保持一致，但其判定能力仍受到观测点设置方式的限制。因此，本文能够判断切片结果在当前观测条件下是否保持准则相关行为，但尚不能覆盖观测点之外的全部潜在语义差异。

其次，实验对象与样例范围仍然有限。Frama-C 与 `dg` 分别代表源码级与 LLVM IR 级两条典型实现路径，但二者仍不足以覆盖程序切片工具的全部设计空间。与此同时，当前样例集合虽然已经揭示出一批稳定问题，但对复杂工业代码中常见的宏展开、外部环境依赖、并发交互以及大规模跨模块传播路径仍覆盖不足。因此，本文结论主要适用于当前代表性工具和实验样例范围内的实证分析，而不能视为对程序切片工具整体行为的穷尽性刻画。

再次，本文对两类工具的结构化分析仍依赖当前的结果归一化和问题归并方式。虽然本文已经区分问题状态统计口径与统一编号问题项统计口径，并通过统一编号、附录清单和外部跟踪信息保持前后对应，但相关结果仍可能受到源码映射精度、人工复核标准以及问题归并策略的影响。因此，本文比较的是在当前归一化与归并口径下两类工具暴露出的结构化问题特征，而不是对不同工具作出一般性的优劣判断。

最后，测试程序来源的覆盖性比较仍有待进一步细化。本文结合 Csmith 随机生成程序与大语言模型生成程序构造测试输入，但两类输入在不同工具上的使用比例和任务规模并不完全一致，尤其是 dg 的测试更多依赖大语言模型生成程序来触发 IR 层面的复杂语义场景。在这种情况下，直接比较 Csmith 与大语言模型生成程序的覆盖率，容易受到样例数量、任务分布和工具适配方式的影响，难以形成完全公平的结论。因此，本文主要从蜕变关系和工具核心模块角度分析覆盖率，而未对两类测试程序来源的覆盖效果进行严格横向比较。

6.3 未来展望

基于本文的实验结果和研究不足，后续工作可以从以下三个方面继续展开。

第一，进一步扩展面向程序切片工具的蜕变关系体系。本文设计的四类关系已经覆盖准则相关语义保持、无关扰动排除和多准则组合一致性等方面，但对于库调用副作用、隐藏状态、并发语义、环境依赖以及复杂别名关系等场景，仍有进一步细化空间。后续可以设计更面向状态传播、环境依赖和复杂别名关系的领域专用关系，以增强对深层语义失效模式的检验能力。

第二，提升输入构造、测试调度与问题归档的自动化程度。一方面，可以围绕大语言模型构建更稳定的提示词模板、约束检查与自动修复流程，提高复杂语义样例生成的可控性；另一方面，也可以将覆盖反馈、历史问题模式与蜕变关系选择结合起来，形成面向高风险实现路径的闭环生成机制。对于已经产生的样例级异常，后续还可以结合异常聚类、相似性分析和历史缺陷档案，对相同或相近根因的问题进行辅助归并，从而降低人工复核成本。

第三，扩展实验对象并细化输入来源比较。后续可以纳入更多不同实现路径的程序切片工具，并构造规模更加均衡、任务分布更加一致的测试程序集合，在

相同工具、相同蜕变关系和相同切片任务配置下，比较 Csmith 随机生成程序与大语言模型生成程序在模块覆盖率、分支覆盖率和问题触发能力上的差异。通过这种方式，可以更系统地分析两类测试输入在覆盖广度、语义深度和缺陷揭示能力上的作用边界，为后续测试输入生成策略的选择与组合提供更充分的实验依据。

参考文献

- [1] WEISER M D. Program Slicing[J/OL]. IEEE Trans. Software Eng., 1984, 10(4): 352-357. <https://doi.org/10.1109/TSE.1984.5010248>. DOI: 10.1109/TSE.1984.5010248.
- [2] GALLAGHER K B, LYLE J R. Using Program Slicing in Software Maintenance[J]. IEEE Transactions on Software Engineering, 1991, 17(8): 751-761. DOI: 10.1109/32.83909.
- [3] AGRAWAL H, HORGAN J R, LONDON S, et al. Fault Localization Using Execution Slices and Dataflow Tests[C]//Proceedings of the Sixth International Symposium on Software Reliability Engineering. IEEE, 1995: 143-151. DOI: 10.1109/ISSRE.1995.497652.
- [4] HORWITZ S, REPS T W. Efficient Comparison of Program Slices[J]. Acta Informatica, 1991, 28(8): 713-732.
- [5] HORWITZ S, REPS T W. The Use of Program Dependence Graphs in Software Engineering[C]//Proceedings of the 14th International Conference on Software Engineering. ACM, 1992: 392-411. DOI: 10.1145/143062.143129.
- [6] PAPOTTI P, TUMA K, MASSACCI F. On the Effects of Program Slicing for Vulnerability Detection during Code Inspection[J]. Empirical Software Engineering, 2025, 30(3): 93. DOI: 10.1007/s10664-025-10636-y.
- [7] AGRAWAL H, HORGAN J R. Dynamic Program Slicing[C]//Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation. ACM, 1990: 246-256. DOI: 10.1145/93542.93576.
- [8] KOREL B, LASKI J. Dynamic Program Slicing[J]. Information Processing Letters, 1988, 29(3): 155-163. DOI: 10.1016/0020-0190(88)90054-3.

- [9] KOREL B, LASKI J. Dynamic Slicing of Computer Programs[J]. Journal of Systems and Software, 1990, 13(3): 187-195. DOI: 10.1016/0164-1212(90)90094-3.
- [10] GUPTA N, SOFFA M L. Hybrid Slicing: An Approach for Refining Static Slices Using Dynamic Information[C]// Proceedings of the 3rd ACM SIGSOFT Symposium on the Foundations of Software Engineering. ACM, 1995: 29-40. DOI: 10.1145/222124.222146.
- [11] GUPTA N, SOFFA M L, HOWARD J. Hybrid Slicing: Integrating Dynamic Information with Static Analysis[J]. ACM Transactions on Software Engineering and Methodology, 1997, 6(4): 370-397. DOI: 10.1145/261640.261644.
- [12] HARMAN M, BINKLEY D, DANICIC S, et al. Amorphous Program Slicing[J]. Journal of Systems and Software, 2003, 68(1): 45-64. DOI: 10.1016/S0164-1212(02)00135-8.
- [13] HARMAN M, HIERONS R M, BINKLEY D, et al. Syntax-Directed Amorphous Slicing[J]. Automated Software Engineering, 2004, 11(1): 27-61. DOI: 10.1023/B:AUSE.00000008667.37988.11.
- [14] AUNG H, VANHOOF W, LIERLER Y, et al. Specialization Slicing[J]. ACM Transactions on Programming Languages and Systems, 2014, 36(2): 5:1-5:67. DOI: 10.1145/2629671.
- [15] TIP F. A Survey of Program Slicing Techniques[J]. Journal of Programming Languages, 1995, 3(3): 121-189.
- [16] BINKLEY D W, GALLAGHER K B. Program Slicing[G]// Advances in Computers: vol. 43. Academic Press, 1996: 1-50. DOI: 10.1016/S0065-2458(08)60641-5.
- [17] XU B, QIAN J, ZHANG X, et al. A Brief Survey of Program Slicing[J]. ACM SIGSOFT Software Engineering Notes, 2005, 30(2): 1-36. DOI: 10.1145/1050849.1050865.

- [18] SILVA J. A Vocabulary of Program Slicing-Based Techniques[J]. ACM Computing Surveys, 2012, 44(3): 12:1-12:41. DOI: 10.1145/2187671.2187674.
- [19] BARRACLOUGH R, HARMAN M, BINKLEY D, et al. A Trajectory-Based Strict Semantics for Program Slicing[J]. Theoretical Computer Science, 2010, 411(11–13): 1372-1386. DOI: 10.1016/j.tcs.2009.10.025.
- [20] 嵇友晴, 卢跃, 潘世文, 等. 基于动态切片与预训练模型的代码漏洞检测[J]. 小型微型计算机系统, 2024, 45(6): 1529-1536.
- [21] ZHANG H, PEI Y, CHEN J, et al. StatFier: Automated Testing of Static Analyzers via Semantic-Preserving Program Transformations[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2023: 237-249. DOI: 10.1145/3611643.3616272.
- [22] HE W, DI P, MING M, et al. Finding and Understanding Defects in Static Analyzers by Constructing Automated Oracles[J]. Proceedings of the ACM on Software Engineering, 2024, 1(FSE): 1656-1678. DOI: 10.1145/3660781.
- [23] BARR E T, HARMAN M, MCMINN P, et al. The Oracle Problem in Software Testing: A Survey[J/OL]. IEEE Trans. Software Eng., 2015, 41(5): 507-525. <https://doi.org/10.1109/TSE.2014.2372785>. DOI: 10.1109/TSE.2014.2372785.
- [24] MCKEEMAN W M. Differential Testing for Software[J]. Digital Technical Journal, 1998, 10(1): 100-107.
- [25] YANG X, CHEN Y, EIDE E, et al. Finding and Understanding Bugs in C Compilers[C]//Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation. ACM, 2011: 283-294. DOI: 10.1145/1993498.1993532.
- [26] LIVINSKII V, BABOKIN D, REGEHR J. Random Testing for C and C++ Compilers with YARPGen[J]. Proceedings of the ACM on Programming Languages, 2020, 4(OOPSLA): 1-25. DOI: 10.1145/3428264.

- [27] LE V, AFSHARI M, SU Z. Compiler Validation via Equivalence Modulo Inputs[C]//Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation. ACM, 2014: 216-226. DOI: 10.1145/2594291.2594334.
- [28] LOPES N P, LEE J, HUR C K, et al. Alive2: Bounded Translation Validation for LLVM[C]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. ACM, 2021: 65-79. DOI: 10.1145/3453483.3454030.
- [29] LI S, THEODORIDIS T, SU Z. Boosting Compiler Testing by Injecting Real-World Code[J]. Proceedings of the ACM on Programming Languages, 2024, 8(PLDI): 156:1-156:23. DOI: 10.1145/3656386.
- [30] JOURDAN J H, LAPORTE V, BLAZY S, et al. A Formally-Verified C Static Analyzer[C]//Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. ACM, 2015: 247-259. DOI: 10.1145/2676726.2676966.
- [31] TANEJA J, LIU Z, REGEHR J. Testing Static Analyses for Precision and Soundness[C]//Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization. ACM, 2020: 81-93. DOI: 10.1145/3368826.3377927.
- [32] CHAROENWET N, STAICU C A, PRADEL M, et al. An Empirical Study of Static Analysis Tools for Secure Code Review[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2024: 691-703. DOI: 10.1145/3650212.3680313.
- [33] PÉREZ S, SILVA J, TAMARIT S. Automatic Testing of Program Slicers[J]. Scientific Programming, 2019, 2019: 1-15. DOI: 10.1155/2019/4108652.
- [34] BINKLEY D W, GOLD N, HARMAN M, et al. ORBS: Language-Independent Program Slicing[C]//Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. ACM, 2014: 109-120. DOI: 10.1145/2635868.2635893.

- [35] STIÉVENART Q, BINKLEY D, DE ROOVER C. An Empirical Evaluation of Quasi-Static Executable Slices[J]. *Journal of Systems and Software*, 2023, 200: 111666. DOI: 10.1016/j.jss.2023.111666.
- [36] CHAI W, YAN R, ZHANG W, et al. Slicing Assisted Program Verification: An Empirical Study[C]// *Lecture Notes in Computer Science: Theoretical Aspects of Software Engineering*: vol. 14777. Springer, 2024: 38-57. DOI: 10.1007/978-3-031-64626-3_3.
- [37] CHEN T Y, CHEUNG S C, YIU S M. Metamorphic Testing: A New Approach for Generating Next Test Cases[R]. Technical Report HKUST-CS98-01. Hong Kong, China: Department of Computer Science, The Hong Kong University of Science, 1998.
- [38] CHEN T Y, TSE T H, ZHOU Z Q. Fault-Based Testing in the Absence of an Oracle[C]// *Proceedings of the 25th Annual International Computer Software and Applications Conference*. 2001: 172-178. DOI: 10.1109/CMPSAC.2001.960614.
- [39] SEGURA S, FRASER G, SÁNCHEZ A B, et al. A Survey on Metamorphic Testing[J]. *IEEE Transactions on Software Engineering*, 2016, 42(9): 805-824. DOI: 10.1109/TSE.2016.2532875.
- [40] CHEN T Y, KUO F C, LIU H, et al. Metamorphic Testing: A Review of Challenges and Opportunities[J]. *ACM Computing Surveys*, 2018, 51(1): 4:1-4:27. DOI: 10.1145/3143561.
- [41] SEGURA S, ZHOU Z. Metamorphic Testing 20 Years Later: A Hands-on Introduction[C]// *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*. ACM, 2018: 538-539. DOI: 10.1145/3183440.3183468.
- [42] SEGURA S, FRASER G, SANCHEZ A B, et al. Metamorphic Testing: Testing the Untestable[J]. *IEEE Software*, 2020, 37(3): 46-53. DOI: 10.1109/MS.2018.2875968.

- [43] MURPHY C, KAISER G, VO I, et al. Properties of Machine Learning Applications for Use in Metamorphic Testing[C]//Proceedings of the 20th International Conference on Software Engineering and Knowledge Engineering. 2008: 867-872.
- [44] XIE X, HO J W K, MURPHY C, et al. Application of Metamorphic Testing to Supervised Classifiers[C]//Proceedings of the 9th International Conference on Quality Software. IEEE, 2009: 135-144. DOI: 10.1109/QSIC.2009.26.
- [45] XIE X, HO J W K, MURPHY C, et al. Testing and Validating Machine Learning Classifiers by Metamorphic Testing[J]. Journal of Systems and Software, 2011, 84(4): 544-558. DOI: 10.1016/j.jss.2010.11.920.
- [46] TOLKSDORF R, LEHMANN D, PRADEL M. Interactive Metamorphic Testing of Debuggers[C]//Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2019: 273-283. DOI: 10.1145/3293882.3330567.
- [47] WINDSOR J, DONALDSON A F, WICKERSON J. High-Coverage Metamorphic Testing of Concurrency Support in C Compilers[J]. Software Testing, Verification and Reliability, 2022, 32(4): e1812. DOI: 10.1002/stvr.1812.
- [48] MANSUR M N, WÜSTHOLZ V, CHRISTAKIS M. Dependency-Aware Metamorphic Testing of Datalog Engines[C]//ISSTA 2023: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2023: 236-247. DOI: 10.1145/3597926.3598052.
- [49] HIRSCH T, HOFER B. Reducing the Length of Dynamic and Relevant Slices by Pruning Boolean Expressions[J]. Electronics, 2024, 13(6): 1146. DOI: 10.3390/electronics13061146.
- [50] CHEN T Y, PUI-LAI P, XIE X. METRIC: METamorphic Relation Identification Based on the Category-Choice Framework[J]. Journal of Systems and Software, 2016, 116: 177-190. DOI: 10.1016/j.jss.2015.07.037.

- [51] CLARK K, BARR E T, HARMAN M, et al. Metamorphic Testing with Causal Graphs[C]//Proceedings of the IEEE International Conference on Software Testing, Verification and Validation. IEEE, 2023: 153-164. DOI: 10.1109/ICST57152.2023.00023.
- [52] LI S, ZHOU Z, CHEN T Y, et al. Metamorphic Relation Generation: State of the Art and Research Directions[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(5): 149:1-149:25. DOI: 10.1145/3708521.
- [53] ZHANG Y, TOWEY D, PIKE M. Automated Metamorphic-Relation Generation with ChatGPT: An Experience Report[C]//Proceedings of the 47th Annual Computers, Software, and Applications Conference. IEEE, 2023: 1780-1785. DOI: 10.1109/COMPSAC57700.2023.00275.
- [54] 孙昌爱, 邢嘉煜, 刘宝莉, 等. 基于路径分析的蜕变测试组生成与优先级排序技术[J]. 计算机学报, 2025, 48(3): 675-693. DOI: 10.11897/SP.J.1016.2025.00675.
- [55] WANG J, HUANG Y, CHEN C, et al. Software Testing With Large Language Models: Survey, Landscape, and Vision[J]. IEEE Transactions on Software Engineering, 2024, 50(4): 911-936. DOI: 10.1109/TSE.2024.3368208.
- [56] XIA C S, PALTENGHI M, TIAN J L, et al. Fuzz4All: Universal Fuzzing with Large Language Models[C]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024). 2024. DOI: 10.1145/3597503.3639121.
- [57] YANG C, DENG Y, LU R, et al. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models[J]. Proceedings of the ACM on Programming Languages, 2024, 8(OOPSLA2). DOI: 10.1145/3689736.
- [58] LI H, HAO Y, ZHAI Y, et al. Assisting Static Analysis with Large Language Models: A ChatGPT Experiment[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23). 2023. DOI: 10.1145/3611643.3613078.

- [59] 李博诺, 王博, 陈明达, 等. 基于大语言模型的 C 程序单元测试方法[J]. 空间控制技术与应用, 2025, 51(1): 115-122. DOI: 10.3969/j.issn.1674-1579.2025.01.012.
- [60] CHEN Y, HU Z, ZHI C, et al. ChatUniTest: A Framework for LLM-Based Test Generation[C]//FSE 2024: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. ACM, 2024: 572-576. DOI: 10.1145/3663529.3663801.
- [61] SHAHANDASHTI K K, MOHAJER M M, BELLE A B, et al. Program Slicing in the Era of Large Language Models[C/OL]//SHAHRIAR H, ALAM K S, OHSAKI H, et al. 49th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2025, Toronto, ON, Canada, July 8-11, 2025. IEEE, 2025: 1224-1233. <https://doi.org/10.1109/COMPSAC65507.2025.00155>. DOI: 10.1109/COMPSAC65507.2025.00155.
- [62] KIRCHNER F, KOSMATOV N, PREVOSTO V, et al. Frama-C: A Software Analysis Perspective[J]. Formal Aspects of Computing, 2015, 27(3): 573-609. DOI: 10.1007/s00165-014-0326-7.
- [63] CHALUPA M. DG: Analysis and Slicing of LLVM Bitcode[C/OL]//HUNG D V, SOKOLSKY O. Lecture Notes in Computer Science: Automated Technology for Verification and Analysis - 18th International Symposium, ATVA 2020, Hanoi, Vietnam, October 19-23, 2020, Proceedings. Springer, 2020: 557-563. https://doi.org/10.1007/978-3-030-59152-6%5C_33. DOI: 10.1007/978-3-030-59152-6_33.
- [64] FERRANTE J, OTTENSTEIN K J, WARREN J D. The Program Dependence Graph and Its Use in Optimization[J]. ACM Transactions on Programming Languages and Systems, 1987, 9(3): 319-349. DOI: 10.1145/24039.24041.
- [65] HORWITZ S, REPS T W, BINKLEY D W. Interprocedural Slicing Using Dependence Graphs[J]. ACM Transactions on Programming Languages and Systems, 1990, 12(1): 26-60. DOI: 10.1145/77606.77608.

- [66] REPS T W, YANG W. The Semantics of Program Slicing and Program Integration[C/OL]//DÍAZ J, OREJAS F. Lecture Notes in Computer Science: TAPSOFT'89: Proceedings of the International Joint Conference on Theory and Practice of Software Development, Barcelona, Spain, March 13-17, 1989, Volume 2: Advanced Seminar on Foundations of Innovative Software Development II and Colloquium on Current Issues in Programming Languages (CC IPL). Springer, 1989: 360-374. https://doi.org/10.1007/3-540-50940-2%5C_47. DOI: 10.1007/3-540-50940-2_47.
- [67] HARROLD M J, MALLOY B A, ROTHERMEL G. Efficient Construction of Program Dependence Graphs[C]//Proceedings of the 1993 International Symposium on Software Testing and Analysis. ACM, 1993: 160-170. DOI: 10.1145/154183.154265.
- [68] LUCIA A D. Program Slicing: Methods and Applications[C]//Proceedings of the First IEEE International Workshop on Source Code Analysis and Manipulation. IEEE Computer Society, 2001: 144-151. DOI: 10.1109/SCAM.2001.972675.
- [69] Frama-C Team. Slicing[EB/OL]. [2026-03-24]. <https://www.frama-c.com/fc-plugins/slicing.html>.
- [70] LLVM Project. Source Level Debugging with LLVM[EB/OL]. [2026-03-24]. <https://llvm.org/docs/SourceLevelDebugging.html>.
- [71] CUOQ P, MONATE B, PACALET A, et al. Testing Static Analyzers with Randomly Generated Programs[C]//Lecture Notes in Computer Science: NASA Formal Methods: vol. 7226. Springer, 2012: 120-125. DOI: 10.1007/978-3-642-28891-3_12.

致 谢

行文至此，我的硕士研究生阶段也即将画上圆满的句号。回首这段时光，既有探索的艰辛，也有成长的喜悦，完成本论文离不开许多老师、同学和亲友的帮助与支持，在此谨以真挚的心意表达我深深的谢意。

首先，我要衷心感谢我的导师钮鑫涛老师。您在研究选题、方法设计、实验推进以及论文撰写的各个环节中给予了耐心细致的指导。导师严谨的学术态度、敏锐的思考视角以及对科研问题的深入洞察，不仅丰富了我在本课题的研究，也深化了我对学术研究的理解。

感谢南京大学智能软件与工程学院提供的良好科研环境和学习平台，使我能够专注于科研任务、汲取先进知识。此外，感谢课题组的同学们，在实验设计、技术讨论和成果分析中给予的热心帮助和宝贵建议，你们的支持让这段科研旅程充满了温暖与动力。

特别感谢开源社区的工具开发者和维护者们，正是这些开源切片工具和技术文档，为本研究提供了可靠的实验基础和技术支持，让我有机会对比分析不同工具的行为和特性。

同时，我也要感谢我的家人和朋友。感谢你们在学习与生活中的理解、陪伴与鼓励，是你们的支持让我在遇到困难时依然坚持前行。

最后，致谢所有在我的学业道路上给予帮助的人。你们的支持让我更加坚定科研探索的信心，这段经历将成为我人生中难忘的记忆。

附录 A 问题项清单

A.1 第 5 章统一编号问题项清单

为支撑第 5 章中的结构分析、案例引用与状态核查，本节集中给出纳入正文统一编号问题项统计口径的 43 个问题项，并同步标注其主归因关系与当前状态。表 A-1 中的编号与正文保持一致，供后续交叉引用与状态核查使用；当前状态仍统一采用“已确认”、“已提交”、“待提交”三类口径，未纳入正文统计口径的设计边界项、显式不支持项与其他中间记录本文不进行列举。

表 A-1 第 5 章统一编号问题项及其主归因关系与状态

编号	工具	主 归 因 关 系	类型	当前状态	问题摘要
1	Frama-C	MR1	语义错误	已确认	等价程序经切片后观测值不一致，未定义求值顺序被线性化。
2	Frama-C	MR1	运行异常	已确认	切片删除关键循环退出条件，重编译后接近死循环。
3	Frama-C	MR1	语义错误	已确认	局部常量数组初始化丢失，但后续读取仍被保留。
4	Frama-C	MR1	运行异常	已确认	指针初始化被切掉，后续解引用稳定崩溃。
5	Frama-C	MR1	编译/链接	已确认	函数重命名后函数指针仍指向旧符号，导致链接失败。
6	Frama-C	MR1	编译/链接	已确认	切片产物保留 <code>__fc_vla_alloc</code> 调用，但缺少可链接实现。
7	Frama-C	MR1	编译/链接	已确认	<code>va_list</code> 被生成为非法数组类型赋值，切片代码无法编译。
8	Frama-C	MR1	语义错误	已确认	初始化表达式被删除，后续使用未初始化值。
9	Frama-C	MR1	运行异常	已确认	除数初始化被切掉，切片程序稳定触发 SIGFPE。
10	Frama-C	MR1	语义错误	已确认	多个局部变量初始化与指针链丢失，导致观测值偏差。
11	Frama-C	MR1	语义错误	已确认	函数切片破坏参数与数据流，运行结果发生变化。
12	Frama-C	MR4	过保留	已确认	与准则无关的全局变量仍被保留。
13	Frama-C	MR3	过保留	已确认	未被修改的准则变量仍被保留，标记传播过于保守。

续表 A-1

编号	工具	主 归 因 关 系	类型	当前状态	问题摘要
14	Frama-C	MR1	编译/链接	已提交	导出切片保留 <code>__fc_atomic_*</code> 调用, 但缺少实现。
15	Frama-C	MR1	语义错误	已提交	<code>constructor</code> 、 <code>destructor</code> 与 <code>atexit</code> 生命周期副作用丢失。
16	Frama-C	MR1	编译/链接	已提交	导出结果引入 <code>pid_t</code> , 严格 C11 下编译失败。
17	Frama-C	MR1	内部异常	已提交	<code>slice-loop-inv</code> 在 <code>term-lval dependency</code> 路径上因 NYI 中止。
18	dg	MR1	语义错误	待提交	<code>memmove</code> 源地址依赖被切掉, 切片结果错误。
19	dg	MR1	语义错误	待提交	<code>strncat</code> 的副作用调用被删除, 切片结果错误。
20	dg	MR1	语义错误	待提交	<code>setjmp/longjmp</code> 相关依赖未被正确保留。
21	dg	MR1	语义错误	待提交	回调逃逸符号依赖丢失, 切片结果错误。
22	dg	MR1	语义错误	待提交	<code>constructor</code> 初始化依赖被切掉。
23	dg	MR1	语义错误	待提交	<code>cmpxchg</code> 本地初始化依赖遗漏。
24	dg	MR1	语义错误	待提交	<code>bcopy</code> 长度相关副作用被切掉。
25	dg	MR1	语义错误	待提交	<code>explicit_bzero</code> 的清零副作用被删除。
26	dg	MR1	语义错误	待提交	<code>memfrob</code> 长度相关副作用被切掉。
27	dg	MR1	语义错误	待提交	<code>qsort</code> 调用及其副作用被删除。
28	dg	MR1	语义错误	待提交	<code>swab</code> 调用被删除。
29	dg	MR1	语义错误	待提交	<code>stpcpy/stpncpy</code> 相关效应跟踪不完整。
30	dg	MR1	语义错误	待提交	宽字符写入类 API 的副作用被删除。
31	dg	MR1	语义错误	待提交	系统 API 输出缓冲区效应被切掉。
32	dg	MR1	语义错误	待提交	<code>errno</code> 语义副作用未被保留。
33	dg	MR1	语义错误	待提交	有状态 API 的副作用未被保留。
34	dg	MR1	内部异常	待提交	<code>indirect goto</code> 相关 IR 或切片过程会崩溃。
35	dg	MR1	语义错误	待提交	内联汇编的内存副作用未被正确建模。
36	dg	MR1	语义错误	待提交	<code>va_copy</code> 相关依赖未被正确保留。
37	dg	MR1	内部异常	已提交	<code>switch</code> 后继关系处理会触发崩溃。
38	dg	MR1	内部异常	已提交	<code>vector insertelement</code> 相关 IR 会触发崩溃。
39	dg	MR1	内部异常	已提交	<code>COMDAT</code> 相关无效 IR 会触发崩溃。
40	dg	MR1	内部异常	已提交	<code>dbg.value</code> 相关 IR 会触发崩溃。
41	dg	MR1	语义错误	已提交	<code>noreturn/unreachable</code> 处理会导致切片错误。
42	dg	MR1	内部异常	待提交	<code>ConstantExpr</code> 指针比较派生值在构图阶段触发 <code>missing value in graph</code> 崩溃。
43	dg	MR3	过保留	待提交	按控制不可达原则构造的死路径负载在切片后仍被保留, 涵盖矛盾条件、零迭代循环与非选中 <code>switch</code> 分支。

A.2 Frama-C 问题项的外部跟踪信息

本节汇总 Frama-C 问题项的外部跟踪信息，并与正文中的统一编号问题项统计口径保持一致。其中，编号 1–13 的当前状态记为“已确认”，可对应到 Frama-C 公共仓库中的 GitLab issue；编号 14–17 的当前状态记为“已提交”，当前统一保留其补充提交档案编号。

表 A-2 Frama-C 已确认问题项的公开 issue 跟踪信息

编号	主 归 因 关 系	当前状态	上游编号	上游状态	公开跟踪信息
1	MR1	已确认	2735	closed	Issue 2735: 等价程序切片后观测值不一致。
2	MR1	已确认	2736	opened	Issue 2736: 关键循环退出条件被切掉，程序接近死循环。
3	MR1	已确认	2737	closed	Issue 2737: 局部常量数组初始化丢失。
4	MR1	已确认	2739	closed	Issue 2739: 指针初始化被删除后发生解引用崩溃。
5	MR1	已确认	2740	closed	Issue 2740: 函数重命名后函数指针仍指向旧符号。
6	MR1	已确认	2741	closed	Issue 2741: 导出切片保留 <code>__fc_vla_alloc</code> 但缺少可链接实现。
7	MR1	已确认	2742	closed	Issue 2742: <code>va_list</code> 被生成为非法数组类型赋值。
8	MR1	已确认	2746	closed	Issue 2746: 初始化表达式被删除。
9	MR1	已确认	2747	closed	Issue 2747: 除数初始化被切掉后触发 SIGFPE。
10	MR1	已确认	2748	closed	Issue 2748: 初始化与指针链依赖同时丢失。
11	MR1	已确认	2749	closed	Issue 2749: 函数切片破坏参数与数据流。
12	MR4	已确认	2750	closed	Issue 2750: 与准则无关的全局变量被保留。
13	MR3	已确认	2751	closed	Issue 2751: 未被修改的准则变量仍被保留。

表 A-3 Frama-C 已提交问题项的补充档案信息

编号	主 归 因 关 系	当前状态	档案编号	档案标识	说明
14	MR1	已提交	2753-007	issues/007-c11-atomics-undefined-symbols	导出切片保留 <code>__fc_atomic_*</code> 调用，但缺少实现。
15	MR1	已提交	2753-010	issues/010-gnu-constructor-dropped	constructor、destructor 与 atexit 生命周期副作用丢失。
16	MR1	已提交	2753-015	issues/015-posix-pid_t-missing	导出结果引入 <code>pid_t</code> ，严格 C11 下编译失败。
17	MR1	已提交	2753-016	issues/016-slicing-term-lval-deps	<code>slice-loop-inv</code> 在 <code>term-lval dependency</code> 路径上因 NYI 中止。

A.3 dg 已提交问题的外部跟踪信息

为保持正文统计口径与外部跟踪状态的一致性，本节汇总 dg 已提交问题的公开跟踪信息。根据当前归档，dg 已向其公共 GitHub 仓库提交 5 个问题，issue 编号范围为 #471 至 #475；这些问题在正文统一编号问题项统计口径中对应编号 37–41。除这 5 项外，其余 dg 问题项在本文问题状态统计口径下统一记为“待提交”。

表 A-4 dg 已提交问题的 GitHub issue 范围

GitHub issue	当前状态	GitHub issue 链接	备注
#471	已提交	Issue 471	待上游确认
#472	已提交	Issue 472	待上游确认
#473	已提交	Issue 473	待上游确认
#474	已提交	Issue 474	待上游确认
#475	已提交	Issue 475	待上游确认

表 A-5 dg 已提交问题对应的统一编号问题项

编号	主 归 因 关系	当前状态	问题摘要	外部跟踪
37	MR1	已提交	switch 后继关系处理会触发崩溃。	GitHub #471–#475
38	MR1	已提交	vector insertelement 相关 IR 会触发崩溃。	GitHub #471–#475
39	MR1	已提交	COMDAT 相关无效 IR 会触发崩溃。	GitHub #471–#475
40	MR1	已提交	dbg.value 相关 IR 会触发崩溃。	GitHub #471–#475
41	MR1	已提交	noreturn/unreachable 处理会导致切片错误。	GitHub #471–#475

附录 B LLM 提示词设计

B.1 LLM 种子程序的提示词模板与主题维度

为提高第 3 章与第 5 章中双轨生成策略的可复现性，本节补充说明用于实验的 LLM 种子程序是如何被引导生成的。具体地，实验阶段对 LLM 的调用并非采用单一固定提示词一次性生成，而是通过“基础提示词 + 关系导向附加约束 + 编译筛选与人工复核”的方式逐步获得可用样例。本节给出实验中稳定使用的提示词模板及其主题维度，以支持后续复现与方法核查。

B.1.1 基础提示词模板

实验中用于生成 LLM 种子程序的基础提示词可概括为如下模板：

请生成一个用于测试静态后向程序切片工具的单文件 C11 程序。程序应可独立编译运行，并包含明确的观测点或输出语句，便于后续设置切片准则与比较切片结果。

请优先围绕<feature_focus>构造高风险语义场景，重点考虑多级指针、函数指针、结构体字段、数组与内存操作等数据相关结构，以及回调、setjmp/longjmp、switch/goto、构造/析构函数、原子操作、变参、VLA 等 C 语言特性，并使<criteria>的值依赖于<dependency_focus>。

请避免把未定义行为作为触发前提，不依赖大型外部工程或复杂构建系统；程序在切片导出后应尽可能保持可重编译性。输出仅包含完整 C 源代码，不要附加解释。

这一模板本身只定义稳定约束，不直接绑定到某一个具体问题。实际使用时，可以在该模板之上叠加与蜕变关系、关注特征和观测准则相关的附加要求。具体地，实验中的提示词并不是把“AST 构造”、“依赖图建模”、“切片计算”、“结果导出”分别写成孤立命令，而是以 C 语言语义特征为显式约束，再以依赖关系、

准则设置和导出后可编译性作为设计目标，从而更自然地引导模型生成可执行、可筛选的候选样例。

B.1.2 关系导向附加约束

在基础提示词之上，本文会根据目标蜕变关系叠加不同的附加要求。表 B-1 总结了这一层的组织方式。

表 B-1 面向四类蜕变关系的 LLM 种子程序附加约束

关系	附加要求	主要目的
MR1	生成观测点清晰、语义边界明确的程序,便于后续构造重命名、等价替换或语义保持变体。	检验准则相关语义保持,并暴露语言边界与导出路径上的脆弱点。
MR2	在不改变准则相关结果的前提下,强化冗余赋值、额外缓冲区、中间指针、拷贝链与无关数据传播。	观察工具是否错误保留无关数据依赖,侧重数据依赖与别名传播路径。
MR3	在不改变准则相关结果的前提下,引入额外分支、switch、goto、短路求值、不可达路径或非局部控制转移。	观察工具是否错误保留无关控制依赖,侧重控制依赖与控制流建模路径。
MR4	要求程序同时提供多个单准则观测点,并能自然组成组合准则,便于比较单准则与联合准则结果。	检验多准则切片的一致性,并考察工具在准则合并时的稳定性。

B.1.3 提示词主题维度

在实验过程中,LLM 提示词的主要组织方式并不是直接要求模型“构建 AST”或“生成 PDG/CDG”,而是通过特定 C 语言特性间接施压相应分析环节。表 B-2 总结了实验中反复使用的主题维度。

其中,C 语言特性通常是提示词中最外层、最显性的部分,因为它最容易被模型直接理解并体现在代码结构上;而 AST 构造、依赖图建模、切片传播和结果导出,则更多体现为这些特性所对应的内部分析环节。也正因为如此,实验中

的提示词大多先从“函数指针 + 回调”、“变参 + 导出”、“switch + 非局部控制转移”、“多级指针 + 内存拷贝”、“构造/析构与全局状态”这类特征组合入手，再由后续筛选保留真正能够进入切片流程的样例。

表 B-2 LLM 种子程序提示词的主题维度及其对应分析环节

主题维度	代表性特征或要求	主要对应分析环节
C 语言特性与前端构造	varargs、VLA、原子操作、位域、volatile、restrict、函数属性、弱符号、内联汇编、constructor/destructor。	语法前端、AST 构造、结果导出与重编译。
指针、数组与数据依赖	多级指针、结构体字段访问、数组索引、指针算术、memcpy/memmove/字符串操作、类型穿透。	别名分析、数据依赖边构建、切片保留/删除决策。
控制流与控制依赖	switch、goto、短路求值、循环控制、死代码包装、setjmp/longjmp、noreturn/unreachable。	控制流图构建、控制依赖传播、控制相关切片规则。
过程间传播与调用边	函数指针、回调、全局状态更新、库调用副作用、初始化与清理函数、副作用型 API。	调用图构建、过程间传播、跨函数切片路径。
准则设置与结果导出	明确的观测点、可组合的多准则、稳定输出位置、切片后可重编译约束。	切片计算、准则合并、结果导出与人工复核。

学位论文出版授权书

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：袁 闯
2026 年 5 月 22 日

论文题名	面向 C 程序切片工具的蜕变测试与自动化缺陷检测研究				
研究生学号	522024720008	所在院系	智能软件与 工程学院	学位年度	2026
论文级别	☐ 学术学位硕士 ☒ 专业学位硕士 ☐ 学术学位博士 ☐ 专业学位博士				
作者 Email	522024720008@smail.nju.edu.cn				
导师姓名	钮鑫涛 助理教授				

论文涉密情况：

☐ 不保密

☐ 保密，保密期（_____年____月____日至_____年____月____日）

